

AA 272 Fall 2025. Homework 4

Colin Skinner, colin.skinner@stanford.edu

Instructors: Prof. Grace Gao, Keidai Iiyama, Guillem Casadesus Vila, and Hannah Nabavi

October 31, 2025

Goal of this problem set: In this problem set, we will tie some loose ends in calculating the pseudorange, and we will see how errors (especially multipath errors) propagate into the code delay tracking. Lastly, we will discuss the Extended Kalman Filter (EKF) and show how we can use ranging information and dynamics together through time to combat errors.

Instructions & Notes

1. This problem set is **due at 4:00 PM on November 14, 2025 via Gradescope**.
2. This homework is scored out of **50 points total** and includes a corresponding self-care component. The point value for each homework question is indicated next to the question number. Note that each homework will be equally weighted in the calculation of your final grade.
3. Please note that homework is intended to supplement and reinforce your learning. We design them to guide you through the process of going deeper into the concepts we cover in class, and beyond.
4. You are free to discuss class contents and these problems with others. However, your submitted solutions and code must be your own work, written from scratch.
5. **AI tools** (e.g., Chat GPT, Gemini) can be viewed as search engines or even classmates. They must not be used to directly solve any of the problems. For example, it is acceptable to discuss general topics about GNSS, edit a plot, or how to solve a linear system using `numpy`.
6. **Ensure that you typeset your solutions.** We encourage using the provided notebooks or LaTeX to type up your solutions, but you are free to use Microsoft Word or other editors.
7. Ensure that your solution includes the plots required by the question, **in the specified format** outlined in the problem for full credit.
8. If you are spending an excessive amount of time on any part of any question, please post on Ed or come to office hours (see Canvas).
9. Please **start early** so you can address any questions you might have, and have fun!

Please submit both a PDF of your solutions and your code via the programming assignment entry on Gradescope.

Problem 1 (8 points): Mathematical Model of the DLL

Goal of this problem: In problem set 2, we generated the PRN codes and demonstrated correlation over the entire 1023 chip sequence. Locating that single chip with the peak is more akin to acquisition. In this problem, students will mathematically assess the impacts of errors on the delay lock loop (DLL).

Recall that our DLL correlator outputs will be affected by noise. In lecture, we represented the early and late correlator outputs as:

$$Z_E = S_E + N_E, \quad Z_L = S_L + N_L,$$

where S_E and N_E are the signal and noise components of the early correlator sample, and S_L and N_L are the signal and noise components of the late correlator sample, in units of square root of power (i.e., $\sqrt{W}$). The τ chip offset that makes $S_E = S_L$ corresponds to the ideal pseudorange (i.e., no errors other than satellite and user clock). However, we cannot directly obtain S_E and S_L due to the noise terms. Instead, we only have access to the outputs Z_E and Z_L .

How will N_E and N_L affect our measured pseudorange? Figure 1 depicts how these noise components affect the early-minus-late DLL discriminator function. We have chosen to plot the $\Delta\tau$ axis centered on the ideal chip offset to assess the error in the pseudorange.

Figure 1.1: Depiction of early-minus-late discriminator function in presence of noise. T_C represents the chip width and C represents the received signal power. We observe that these noise terms lead to a vertical shifting of the discriminator function, which results in a shifted “zero crossing” for the discriminator and a corresponding error in the pseudorange. Note that this discriminator is plotted for a correlator spacing of $d = 1$ chip.

The noise model in this problem is a good model of some noise, such as thermal noise or error sources that change the variance in the pseudorange. Thinking back to problem 2 on homework 2, we can use the intuition from this model for selecting appropriate weights in weighted least squares. However, this model does not capture the uncorrected delays in signal reception that induce biases in the measured pseudorange. In particular, the shape of the discriminator above is not a good model for multipath, which is what the next problem will handle. That said, this model will take you a long way in understanding pseudorange uncertainty.

1.1 Pseudorange Error [2 points]

Using Figure 1.1 as a reference, show step-by-step how we can derive an expression for the induced pseudorange error **in meters** in terms of N_E and N_L . Clearly indicate your final expression as well.

Hint: Lecture slides may prove helpful to check your final answer.

Hint: N_E and N_L are in units of $\sqrt{W}$. T_C is in units of seconds.

Representing the error as τ in meters. Since the pseudorange error is in units of seconds, it must be multiplied by the speed of light to correct its units.

$$slope = \frac{rise}{run} = \frac{N_E - N_L}{c\tau} = \frac{N_E - N_L + \sqrt{C}}{c\frac{T_c}{2} + \tau}$$

$$\frac{T_c}{2}(N_E - N_L) + c\tau(N_E - N_L) = (N_L - N_L + \sqrt{C})c\tau$$

$$\frac{T_c}{2}(N_E - N_L) = \sqrt{C}c\tau$$

$$\tau = \frac{cT_C(N_E - N_L)}{2\sqrt{(C)}}$$

1.2 Probability Distribution [4 points]

Suppose that N_E and N_L come from a zero-mean Gaussian distribution with variance σ_N^2 , i.e., $N_E, N_L \sim \mathcal{N}(0, \sigma_N^2)$. Using the expression found in part (a), derive the probability distribution for the corresponding pseudorange error (keep your terms **in meters**).

Hint 1: For an early-minus-late correlator spacing of $d = 1$ chip (as shown in Figure 1.1), we can assume that N_E and N_L are uncorrelated.

Hint 2: If a random variable is Gaussian, you only need to compute the mean μ_ρ and standard deviation σ_ρ to define its distribution.

Hint 3: Recall that the mean is defined as $\mu_\rho := \mathbb{E}[\text{pseudorange error}]$ and the variance is defined as $\sigma_\rho^2 := \mathbb{E}[(\text{pseudorange error} - \mu_\rho)^2]$.

Hint 4: Linear combinations of Gaussian random variables is still Gaussian. So, you will not need to do any integrals for this problem, just some algebraic work.

ANSWER

Adding gaussian distributions adds the means and the standard distribution. Assuming $\sqrt{C}$ is a constant, the resulting pseudorange error distribution is as follows:

$$\tau = \frac{cT_C}{2\sqrt{(C)}}(N_E - N_L)$$

$$\tau = \frac{cT_C}{2\sqrt{(C)}}(N(0, \sigma_N^2) - N(0, \sigma_N^2))$$

$$\tau = \frac{cT_C}{2\sqrt{(C)}}N(0, 2\sigma_N^2)$$

$$\sigma_\rho^2 = 2\sigma_N^2 * \left(\frac{cT_C}{2\sqrt{(C)}} \right)^2$$

$$\sigma_\rho^2 = 2\sigma_N^2 \frac{c^2 T_c^2}{4C}$$

$$\sigma_\rho^2 = \frac{\sigma_N^2 c^2 T_c^2}{2C}$$

1.3 Pseudorange Precision [1 point]

For reasons beyond the current discussion, we can model the variance σ_N^2 via

$$\sigma_N^2 = \frac{N_0}{2T}$$

where N_0 is the noise density ratio (in units of power per frequency or W/Hz) and T is the averaging time (in units of seconds). In problem 3 of homework 3, we were correlating across 1023 chips. However, we could have continued correlating beyond 1023 chips using the periodic nature of the PRN codes. Correlating or averaging over a longer amount of time will diminish the impact of the noise and give us a clearer peak. You may want to revisit the lecture slides, with attention to the elapsed time, to see this in action.

The quantity carrier-to-noise-density ratio (C/N_0) is the quantity we referred to loosely as signal strength in the lectures, and the GNSS Logger App. Most people (including the developers of the GNSS Logger App and many chipset manufacturers) will report C/N_0 in decibel-Hertz (dB-Hz), whereas the subsequent calculations are more straightforward without the decibel conversion such that C/N_0 has units of Hertz. A nice property of C/N_0 is that we can estimate it using the measured power in the received signal and the correlation process we already do as part of CDMA.

Plug in this expression of σ_N^2 into your standard deviation equation above to derive the change in pseudorange precision (i.e., σ_ρ) based on the C/N_0 . Please isolate the term C/N_0 in parentheses for readability.

ANSWER

$$\tau = N \left(0, \frac{\sigma_N^2 c^2 T_c^2 N_0}{2C} \right)$$

$$\sigma_\rho^2 = \frac{c^2 T_c^2 N_0}{2C} \frac{1}{2T}$$

$$\sigma_\rho^2 = \frac{c^2 T_c^2}{4T} (C/N_0)^{-1}$$

1.4 Tracking Performance [1 point]

Given that the military GPS L1 P(Y) signal's chipping rate is 10 times faster than the civilian GPS L1 C/A, how will the pseudorange error and tracking performance of the military signal compare with that of the civilian signal? Use your derived expression from part (2) to justify your conclusions.

Hint: You will want to look at standard deviation **not variance** since standard deviation is in units of meters, but variance is in units of meters squared.

Side-note: The modernized GPS L5 and Galileo E5 civilian signals also have a 10 times faster chipping rate, but it is transmitted in a different band (namely, the L5/E5 band).

ANSWER

Because the military signal has a 10 times faster chipping rate, the chirp period is 10 times smaller. From our σ_ρ formula, $\sigma_\rho \propto T_c$, so the pseudorange error standard deviation decreases by a factor of 10, improving tracking precision.

Problem 2 (14 points + 2 bonus): DLL & Multipath

Goal of this problem: In the lectures, we discussed how the Delay Lock Loop (DLL) tracks the GPS signal and the presence of multipath errors. In this problem, students will explore how a multipath impacts the DLL in more depth.

When analyzing the DLL discriminator and tracking performance, we often consider an “ideal” auto-correlation peak. Recall our ideal correlation peak:

Figure 2.1: Depiction of ideal auto-correlation function. Correlator spacing d defines the spacing (in chips) between the early and the late code replicas. C is our received signal power.

Notice that this ideal auto-correlation peak in Figure 2.1 has a peak value of $\sqrt{C}$ at a delay of 0 chips, and correlation values of 0 at any delay $\Delta\tau$ which goes beyond ± 1 chips, i.e. for all delays such that $|\Delta\tau| \geq 1$ chips. Also, note that the correlation spacing between the early and late replica is defined as d (in chips).

2.1 Ideal Autocorrelation [2 points]

Write a function `ideal_autocorr(dt)` which takes in any code offset `dt` (in chips) and outputs the corresponding ideal auto-correlation value, as depicted in Figure 2.1. You can consider $C = 1$, such that the correlation peak height is +1.

Using your function, plot the ideal auto-correlation over a range of ± 3 chips offset (i.e., x -axis spans ± 3 chips). Validate that it looks like the plot in Figure 2.1 when `dt` (i.e., $\Delta\tau$) equals zero. For all plots, ensure that your axes are labeled and include grid lines.

Hint: We are *not* considering the auto-correlation sidepeaks in this problem, so you *do not* need to generate the PRN sequence.

ANSWER

```
[146]: import numpy as np
import matplotlib.pyplot as plt

# Lecture 11
def eval_ideal_autocorr(dt, center = 0):
    # Left side: x - dt + 1
    # Right side: -x + dt + 1

    z = np.zeros_like(dt)

    z = np.where(np.abs(dt - center) < 1, 1 - np.abs(dt-center), 0)

    return z

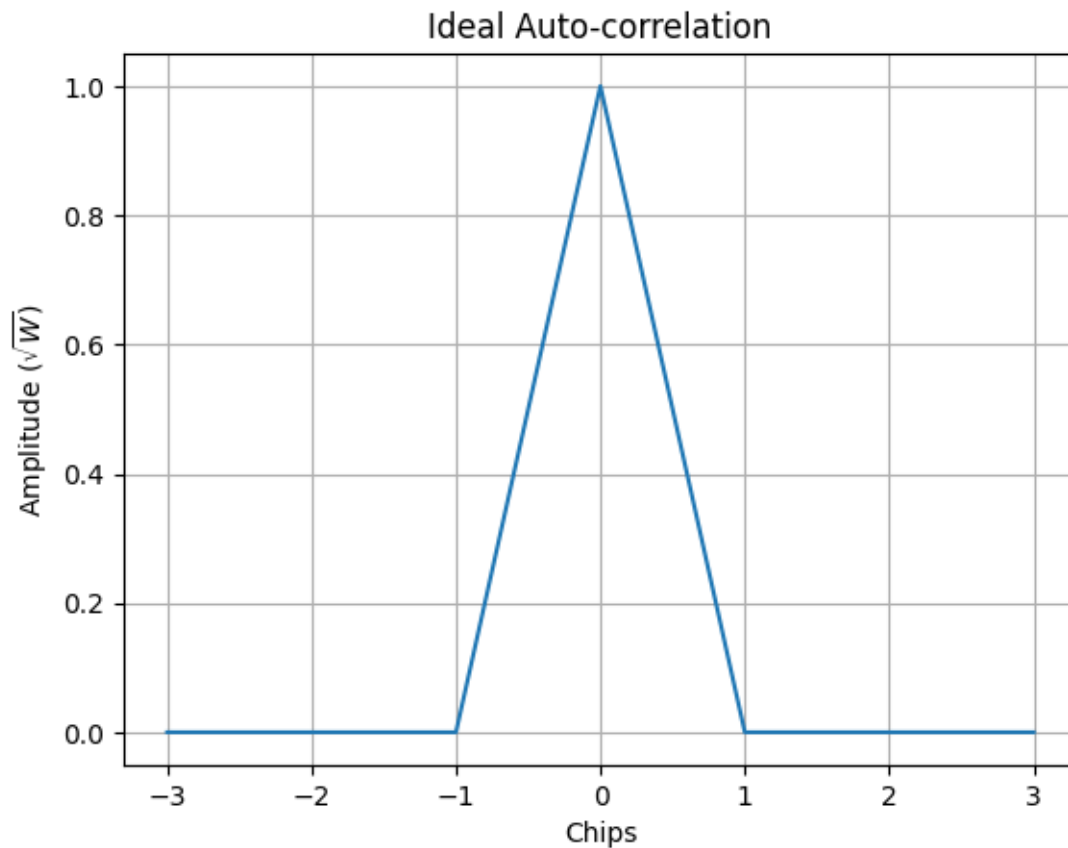
dt = np.linspace(-3, 3, 301)
z1 = eval_ideal_autocorr(dt, 0.5)
z2 = eval_ideal_autocorr(dt, -0.5)
```

```

z = eval_ideal_autocorr(dt)

plt.xlabel("Chips")
plt.ylabel(r"Amplitude ( $\sqrt{W}$ )")
plt.title("Ideal Auto-correlation")
plt.plot(dt, z)
plt.grid()

```



```

[147]: # *** DO NOT MODIFY THE CODE BELOW ***
print("@P2.1\n", *np.round(z, 2))

```

```

@P2.1
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.02 0.04 0.06 0.08 0.1 0.12 0.14 0.16 0.18 0.2 0.22 0.24 0.26 0.28 0.3 0.32
0.34 0.36 0.38 0.4 0.42 0.44 0.46 0.48 0.5 0.52 0.54 0.56 0.58 0.6 0.62 0.64
0.66 0.68 0.7 0.72 0.74 0.76 0.78 0.8 0.82 0.84 0.86 0.88 0.9 0.92 0.94 0.96

```

```

0.98 1.0 0.98 0.96 0.94 0.92 0.9 0.88 0.86 0.84 0.82 0.8 0.78 0.76 0.74 0.72 0.7
0.68 0.66 0.64 0.62 0.6 0.58 0.56 0.54 0.52 0.5 0.48 0.46 0.44 0.42 0.4 0.38
0.36 0.34 0.32 0.3 0.28 0.26 0.24 0.22 0.2 0.18 0.16 0.14 0.12 0.1 0.08 0.06
0.04 0.02 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0

```

2.2 DLL Discriminator [2 points]

Now that we have our ideal autocorrelation peak, we can also build the early-minus-late DLL discriminator from lecture 10 and Problem 1. Specifically, we will start with a correlator spacing of $d = 1$ chip. This means the early replica is 0.5 chips earlier than the prompt replica, and the late replica is 0.5 chips later than the prompt replica.

Plot the DLL discriminator (i.e., $Z_E - Z_L$) output for a correlator spacing of $d = 1$ chip as a function of the code delay over a range of ± 3 chips offset. Use your function from part (1) to create this discriminator output curve, and ensure your plot matches the discriminator plot we observed in lecture 10 and Problem 1.

Hint 1: You can get Z_E and Z_L using your `ideal_autocorr(dt)` function. You just need to shift the input by ± 0.5 chips depending on Z_E and Z_L .

Hint 2: The early replica is highest when the prompt replica is at 0.5 chips. The late replica is highest when the prompt replica is at -0.5 chips.

Hint 3: In the *ideal case*, $N_E = N_L = 0$.

Hint 4: Can you identify the $\pm \frac{T_C}{2}$ peaks of the discriminator? Does the magnitude of the peak match Figure 2 in Problem 1?

ANSWER

[Plot the ideal DLL discriminator for $d=1$ chip.]

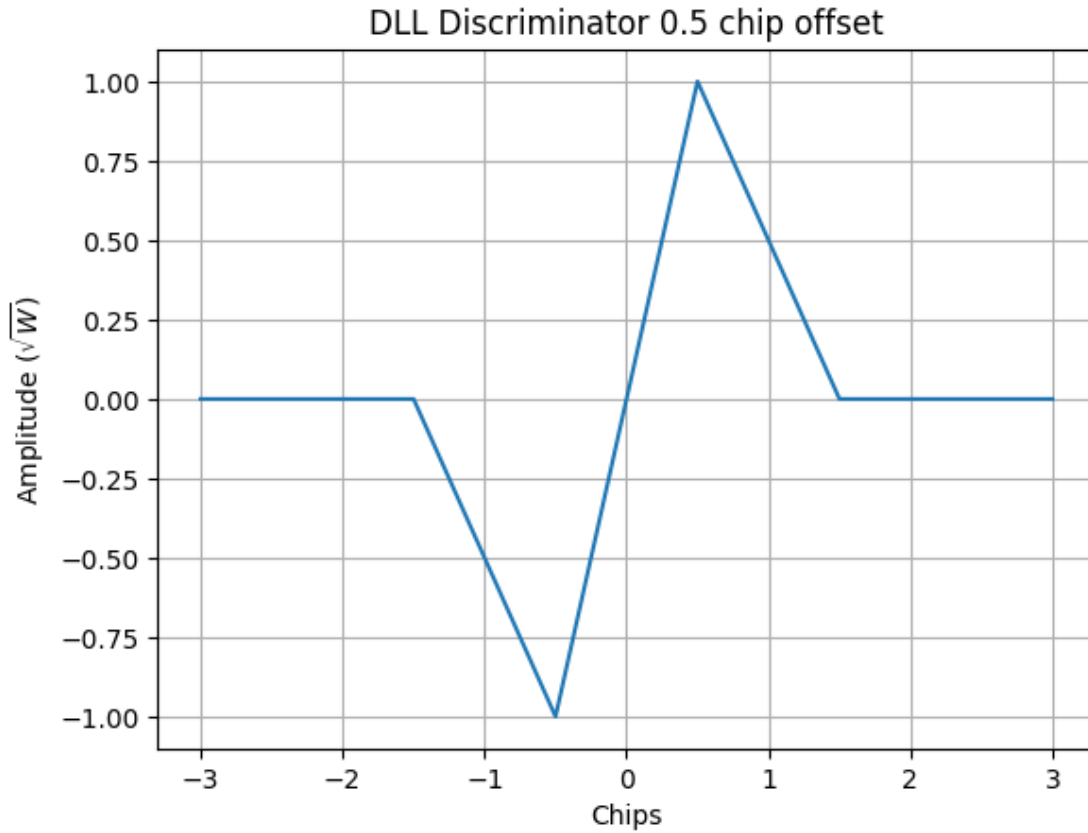
```

[148]: dt = np.linspace(-3, 3, 301)
ze = eval_ideal_autocorr(dt, 0.5)
zl = eval_ideal_autocorr(dt, -0.5)
ze_zl = ze-zl

plt.xlabel("Chips")
plt.ylabel(r"Amplitude ( $\sqrt{W}$ )")
plt.title("DLL Discriminator 0.5 chip offset")
plt.plot(dt, ze_zl)
plt.grid()

# print(ze)
# print(zl)

```



```
[149]: # *** DO NOT MODIFY THE CODE BELOW ***
print("@P2.2\n", *np.round(ze_z1, 2))
```

```
@P2.2
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 -0.02 -0.04
-0.06 -0.08 -0.1 -0.12 -0.14 -0.16 -0.18 -0.2 -0.22 -0.24 -0.26 -0.28 -0.3 -0.32
-0.34 -0.36 -0.38 -0.4 -0.42 -0.44 -0.46 -0.48 -0.5 -0.52 -0.54 -0.56 -0.58 -0.6
-0.62 -0.64 -0.66 -0.68 -0.7 -0.72 -0.74 -0.76 -0.78 -0.8 -0.82 -0.84 -0.86
-0.88 -0.9 -0.92 -0.94 -0.96 -0.98 -1.0 -0.96 -0.92 -0.88 -0.84 -0.8 -0.76 -0.72
-0.68 -0.64 -0.6 -0.56 -0.52 -0.48 -0.44 -0.4 -0.36 -0.32 -0.28 -0.24 -0.2 -0.16
-0.12 -0.08 -0.04 0.0 0.04 0.08 0.12 0.16 0.2 0.24 0.28 0.32 0.36 0.4 0.44 0.48
0.52 0.56 0.6 0.64 0.68 0.72 0.76 0.8 0.84 0.88 0.92 0.96 1.0 0.98 0.96 0.94
0.92 0.9 0.88 0.86 0.84 0.82 0.8 0.78 0.76 0.74 0.72 0.7 0.68 0.66 0.64 0.62 0.6
0.58 0.56 0.54 0.52 0.5 0.48 0.46 0.44 0.42 0.4 0.38 0.36 0.34 0.32 0.3 0.28
0.26 0.24 0.22 0.2 0.18 0.16 0.14 0.12 0.1 0.08 0.06 0.04 0.02 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
```

0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0

2.3 Different Spacings [2 points]

Repeat part (2) for $d = 0.5$ chips. Overlay this DLL discriminator on the DLL discriminator from part (2).

Briefly comment on why the shape is different for a tighter correlator spacing using what you know about the ideal auto-correlation function.

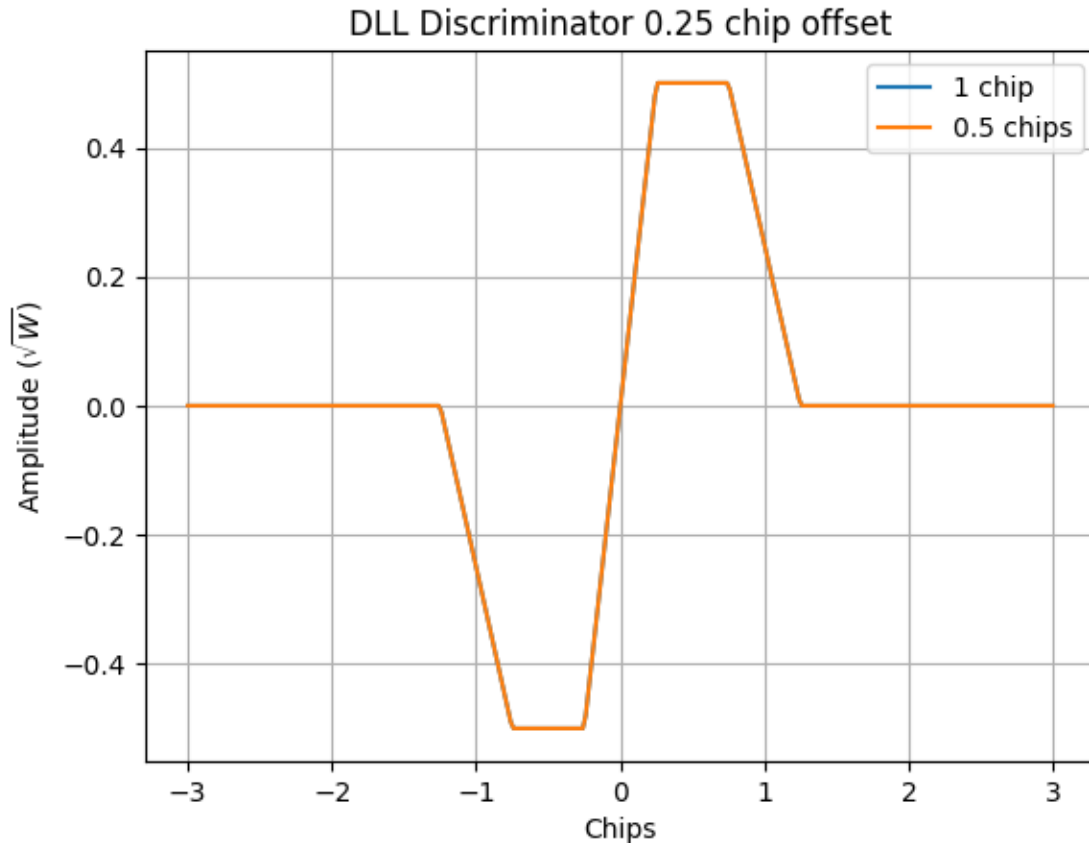
ANSWER

[Plot the DLL discriminators for $d=1$ and $d=0.5$ chips on the same plot and discuss the differences.]

```
[150]: ze = eval_ideal_autocorr(dt, 0.25)
      z1 = eval_ideal_autocorr(dt, -0.25)
      ze_z1 = ze-z1

      plt.xlabel("Chips")
      plt.ylabel(r"Amplitude ( $\sqrt{W}$ )")
      plt.title("DLL Discriminator 0.25 chip offset")
      plt.plot(dt, ze_z1, label="1 chip")
      plt.plot(dt, ze_z1_05, label="0.5 chips")
      plt.legend()
      plt.grid()

      # print(ze)
      # print(z1)
```



We want ZE and ZL to be symmetric on both sides of the autocorrelation triangle for our code tracking error to be 0. Therefore, we take the difference in the two distributions. Specifically, we look at the extrema of the graph. If the dt offset is smaller, then the peaks are smaller because the difference between ZE and ZL is smaller.

```
[151]: # *** DO NOT MODIFY THE CODE BELOW ***
print("@P2.3\n", *np.round(ze_zl, 2))
```

```
@P2.3
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 -0.01 -0.03 -0.05 -0.07 -0.09 -0.11 -0.13 -0.15
-0.17 -0.19 -0.21 -0.23 -0.25 -0.27 -0.29 -0.31 -0.33 -0.35 -0.37 -0.39 -0.41
-0.43 -0.45 -0.47 -0.49 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5
-0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.48
-0.44 -0.4 -0.36 -0.32 -0.28 -0.24 -0.2 -0.16 -0.12 -0.08 -0.04 0.0 0.04 0.08
0.12 0.16 0.2 0.24 0.28 0.32 0.36 0.4 0.44 0.48 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5
0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.49 0.47
0.45 0.43 0.41 0.39 0.37 0.35 0.33 0.31 0.29 0.27 0.25 0.23 0.21 0.19 0.17 0.15
```

```

0.13 0.11 0.09 0.07 0.05 0.03 0.01 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0

```

2.4 Correlation With Multipath [2 points]

Let us consider a simulated multipath scenario and see what happens to our DLL discriminator outputs. Multipath occurs when we have a secondary, delayed peak in addition to the main correlation peak. Generally, the second peak has a weaker received power. But, the second peak could have a positive or negative sign based on the nature of the reflection and the user antenna. The positive case is akin to constructive interference, and the negative case is akin to destructive interference. For the next few questions, let us consider the following multipath scenario, where the delayed peak has a relative delay of 1.25 chips and has half the correlation amplitude as the ideal peak. The 1.25 chips delay is roughly 366 m for the GPS L1 C/A 1.023 MHz chipping rate.

Figure 2.2: Depiction of auto-correlation function in a multipath scenario. The tan curve represents the ideal peak, while the light red curve represents the delayed, reflected peak. Note that the receiver observes the *summation* of these two correlation peaks.

In the presence of multipath, the resulting correlation observed at the receiver (before reaching the DLL) is the *summation* of the two correlation peaks, thereby distorting the resulting DLL discriminator output curve.

Let us model the distorted auto-correlation for this multipath scenario. Create a function called `eval_multipath_autocorr(dt)` which takes in any code offset `dt` (in chips) and outputs the resulting, distorted correlation peak caused by the secondary reflected signal in Figure 2.2.

Using your function and your function from part (1), plot the following *on the same plot*: * Ideal auto-correlation peak * Secondary, delayed peak * Combined distorted auto-correlation

ANSWER

```

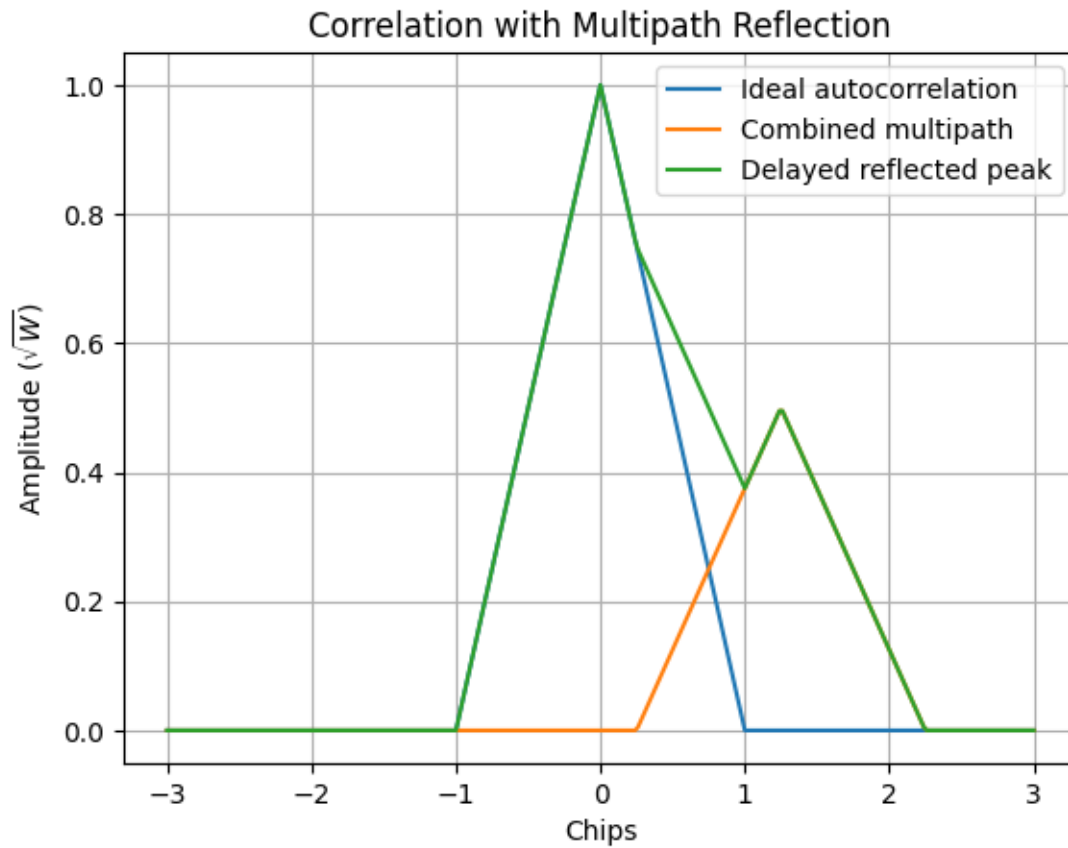
[152]: def eval_multipath_autocorr(dt, amplitude, delay):
        # TODO: Implement
        return eval_ideal_autocorr(dt, delay) * amplitude

z_ideal = eval_ideal_autocorr(dt)
z_multipath = eval_multipath_autocorr(dt, 0.5, 1.25)
z_combined = z + z_multipath # TODO: Compute

plt.xlabel("Chips")
plt.ylabel(r"Amplitude ( $\sqrt{W}$ )")
plt.title("Correlation with Multipath Reflection")
plt.plot(dt, z, label="Ideal autocorrelation")
plt.plot(dt, z_multipath, label="Combined multipath")
plt.plot(dt, z_combined, label="Delayed reflected peak")
plt.grid()
plt.legend()

```

```
plt.show()
```



When a smaller delayed peak is introduced, this causes ZL to be perceived as larger than it actually is (since the reflected peak is also late). Even if ZE and ZL are equal, this does not cause them to be perceived as such.

```
[153]: # *** DO NOT MODIFY THE CODE BELOW ***
print("@P2.4\n", *np.round(z_combined, 2))
```

@P2.4

```
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.02 0.04 0.06 0.08 0.1 0.12 0.14 0.16 0.18 0.2 0.22 0.24 0.26 0.28 0.3 0.32
0.34 0.36 0.38 0.4 0.42 0.44 0.46 0.48 0.5 0.52 0.54 0.56 0.58 0.6 0.62 0.64
0.66 0.68 0.7 0.72 0.74 0.76 0.78 0.8 0.82 0.84 0.86 0.88 0.9 0.92 0.94 0.96
0.98 1.0 0.98 0.96 0.94 0.92 0.9 0.88 0.86 0.84 0.82 0.8 0.78 0.76 0.74 0.73
0.72 0.71 0.7 0.7 0.68 0.68 0.66 0.66 0.64 0.64 0.62 0.62 0.6 0.6 0.58 0.57 0.56
```

```

0.55 0.54 0.53 0.52 0.51 0.5 0.49 0.48 0.47 0.46 0.46 0.45 0.44 0.43 0.42 0.4
0.4 0.38 0.38 0.39 0.4 0.41 0.42 0.42 0.44 0.44 0.46 0.46 0.48 0.48 0.5 0.5 0.48
0.48 0.46 0.46 0.44 0.44 0.42 0.42 0.4 0.4 0.38 0.38 0.36 0.36 0.34 0.34 0.32
0.31 0.31 0.29 0.29 0.27 0.27 0.25 0.25 0.23 0.23 0.21 0.21 0.19 0.19 0.17 0.17
0.15 0.15 0.13 0.12 0.11 0.1 0.09 0.08 0.07 0.06 0.06 0.04 0.04 0.02 0.02 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0

```

2.5 DLL Discriminator With Multipath [4 points]

How would our DLL handle this distorted auto-correlation from multipath? Similar to part (3), plot the DLL discriminator function output for correlator spacings of $d = 0.5$ -chips and $d = 1$ -chip as a function of the code delay on the same plot.

Create a second plot that is zoomed in on the zero-crossing of the discriminator curves (near $\Delta\tau = 0$ chips).

Note: This discrimination curve looks quite different from problem 1 over a ± 3 chips offset, but *resembles* problem 1 near the true zero-crossing.

Note: Recall that the multipath delay was 1.25 chips away or 366 m delay. Even though the delay is quite physically long, it can still noticeably distort our DLL tracking and pseudorange measurement.

ANSWER

```

[154]: def dll_discriminator(center):
        return eval_ideal_autocorr(dt, center) + eval_multipath_autocorr(dt, 0.5,
        ↪center + 1.25)

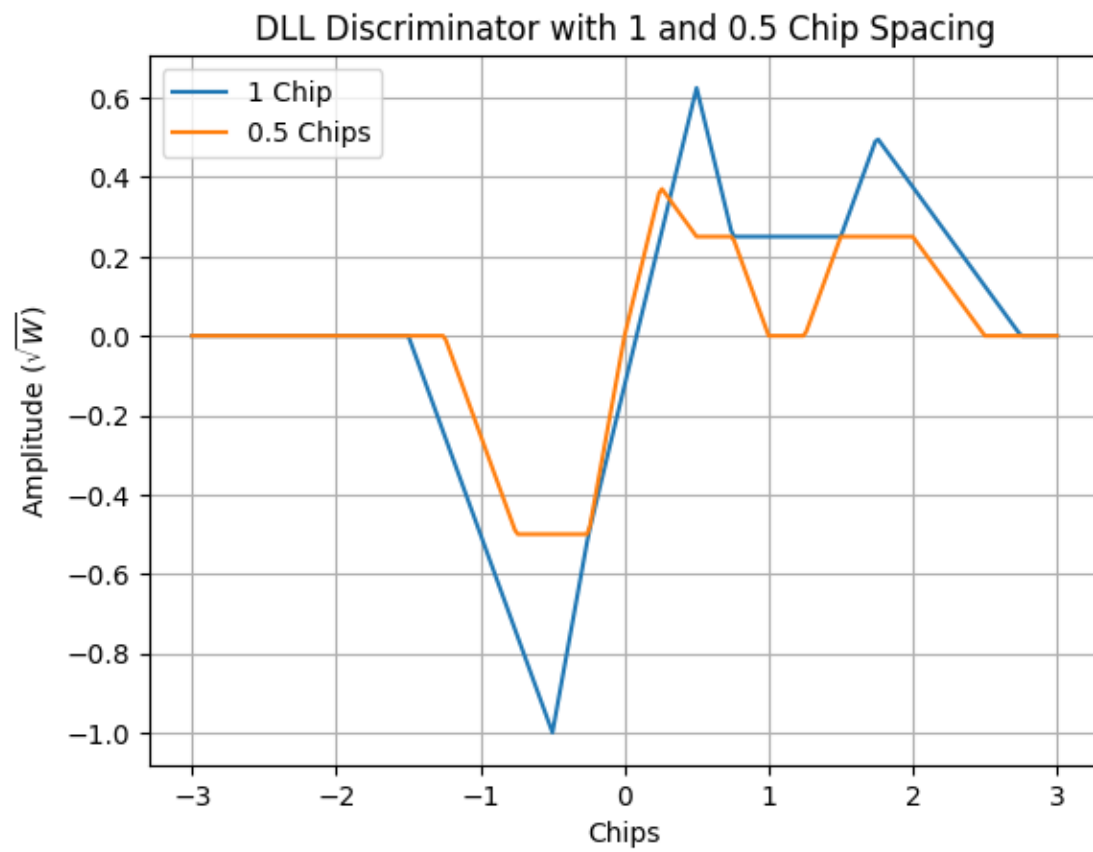
ze_zl_1chip = dll_discriminator(0.5) - dll_discriminator(-0.5) # TODO: Compute
ze_zl_05chip = dll_discriminator(0.25) - dll_discriminator(-0.25) # TODO:
        ↪Compute

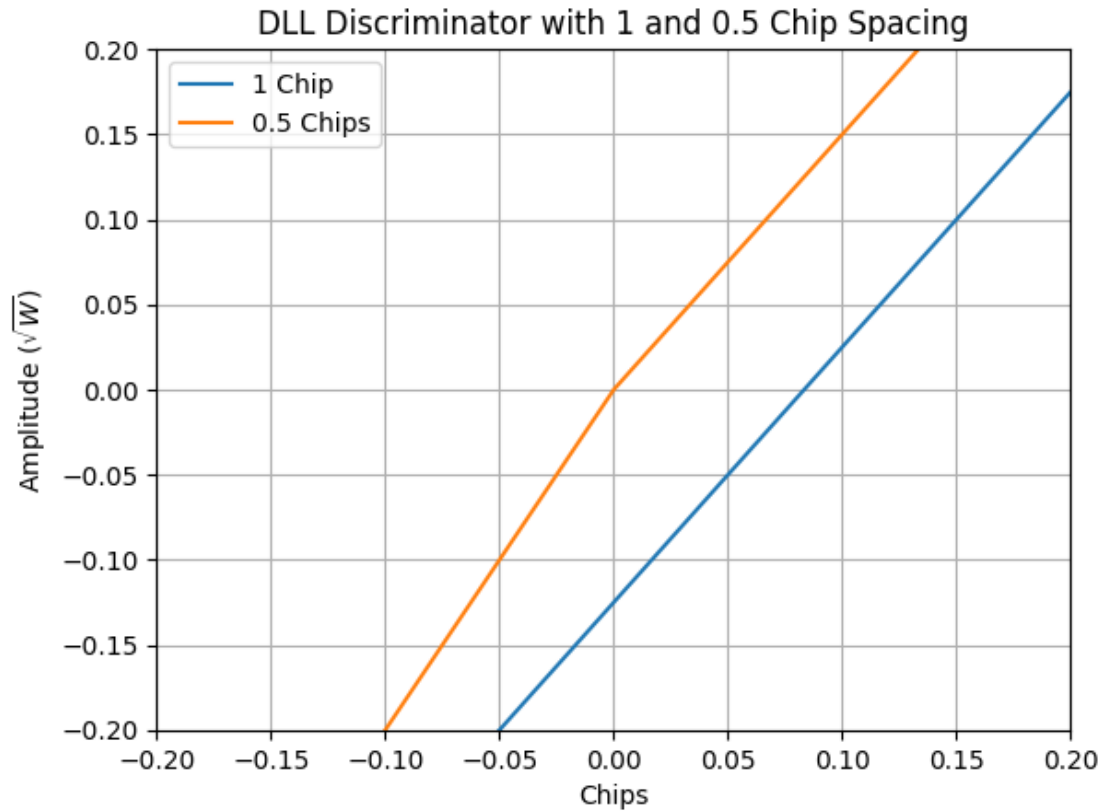
plt.xlabel("Chips")
plt.ylabel(r"Amplitude ( $\sqrt{W}$ )")
plt.title("DLL Discriminator with 1 and 0.5 Chip Spacing")
plt.plot(dt, ze_zl_1chip, label="1 Chip")
plt.plot(dt, ze_zl_05chip, label="0.5 Chips")
plt.grid()
plt.legend()
plt.show()

plt.xlabel("Chips")
plt.ylabel(r"Amplitude ( $\sqrt{W}$ )")
plt.title("DLL Discriminator with 1 and 0.5 Chip Spacing")
plt.plot(dt, ze_zl_1chip, label="1 Chip")
plt.plot(dt, ze_zl_05chip, label="0.5 Chips")
plt.xlim(-0.2, 0.2)
plt.ylim(-0.2, 0.2)

```

```
plt.grid()
plt.legend()
plt.show()
```





```
[155]: dt_small = dt[(dt < 0.2) & (dt >= -0.2)]
ze_zl_05chip_shaved = ze_zl_05chip[(dt < 0.2) & (dt >= -0.2)]
ze_zl_1chip_shaved = ze_zl_1chip[(dt < 0.2) & (dt >= -0.2)]
```

```
def find_x_int_pos_slope(x, y):
    """Based on similar triangles"""

    y_low = y[y<=0][-1]
    y_high = y[y>=0][0]
    x_low = x[y<=0][-1]
    x_high = x[y>=0][0]

    if x_high == x_low:
        return x_high

    dx = x_high-x_low
    dy = y_high-y_low
    return x_low + dx - (dx * y_high/dy)

# y intercept should be in the middle
n = len(dt)
```

```
Y-intercept of 0.5 chips: 0.0
Y-intercept of 1 chips: -0.125
X-intercept of 0.5 chips: 0.0
X-intercept of 1 chips: 0.08333333333333333
```

```
@P2.5
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 -0.02 -0.04
-0.06 -0.08 -0.1 -0.12 -0.14 -0.16 -0.18 -0.2 -0.22 -0.24 -0.26 -0.28 -0.3 -0.32
-0.34 -0.36 -0.38 -0.4 -0.42 -0.44 -0.46 -0.48 -0.5 -0.52 -0.54 -0.56 -0.58 -0.6
-0.62 -0.64 -0.66 -0.68 -0.7 -0.72 -0.74 -0.76 -0.78 -0.8 -0.82 -0.84 -0.86
-0.88 -0.9 -0.92 -0.94 -0.96 -0.98 -1.0 -0.96 -0.92 -0.88 -0.84 -0.8 -0.76 -0.72
-0.68 -0.64 -0.6 -0.56 -0.52 -0.48 -0.45 -0.42 -0.4 -0.37 -0.34 -0.31 -0.28
-0.25 -0.22 -0.19 -0.16 -0.12 -0.09 -0.06 -0.03 -0.0 0.03 0.06 0.09 0.12 0.15
0.18 0.21 0.24 0.27 0.3 0.33 0.36 0.38 0.41 0.44 0.47 0.5 0.53 0.56 0.6 0.62 0.6
0.56 0.53 0.5 0.47 0.44 0.41 0.38 0.35 0.32 0.29 0.26 0.25 0.25 0.25 0.25 0.25
0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.25 0.27 0.29 0.31 0.33 0.35 0.37 0.39 0.41 0.43 0.45 0.47 0.49 0.5 0.48 0.48
0.46 0.46 0.44 0.44 0.42 0.42 0.4 0.4 0.38 0.38 0.36 0.36 0.34 0.34 0.32 0.31
0.31 0.29 0.29 0.27 0.27 0.25 0.25 0.23 0.23 0.21 0.21 0.19 0.19 0.17 0.17 0.15
0.15 0.13 0.12 0.11 0.1 0.09 0.08 0.07 0.06 0.06 0.04 0.04 0.02 0.02 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 -0.01 -0.03 -0.05 -0.07 -0.09 -0.11 -0.13 -0.15
-0.17 -0.19 -0.21 -0.23 -0.25 -0.27 -0.29 -0.31 -0.33 -0.35 -0.37 -0.39 -0.41
-0.43 -0.45 -0.47 -0.49 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5
-0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.5 -0.48
-0.44 -0.4 -0.36 -0.32 -0.28 -0.24 -0.2 -0.16 -0.12 -0.08 -0.04 0.0 0.03 0.06
0.09 0.12 0.15 0.18 0.21 0.24 0.27 0.3 0.33 0.36 0.37 0.36 0.35 0.34 0.33 0.32
0.31 0.3 0.29 0.28 0.27 0.26 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
```

```

0.25 0.25 0.25 0.24 0.22 0.2 0.18 0.16 0.14 0.12 0.1 0.08 0.06 0.04 0.02 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.15 0.17 0.19 0.21 0.23 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25
0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.24
0.23 0.22 0.21 0.2 0.19 0.18 0.17 0.16 0.15 0.14 0.13 0.12 0.11 0.1 0.09 0.08
0.07 0.06 0.05 0.04 0.03 0.02 0.01 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0

```

2.6 Induced Biases [2 points]

For which DLL will the multipath peak(s) induce a bias(es) in our pseudorange error? If so, approximately how much (in meters to a reasonable number of significant figures)? If not, explain why not.

Note: This advantage comes at a cost. Indeed, you may have heard the saying that “there’s no free lunch” in engineering since our design choices often have a trade-off (although, the same saying rarely applies to engineering college campuses, where free lunches and meals are quite commonplace). Explore this further in Bonus 2!

ANSWER

The pseudorange will be affected by any multipath peaks, but in particular, those that are further apart and more likely to have the reflected peak affect only one of the correlators. When ZE and ZL are closer together, a reflected signal is more likely to affect both at once, but when they are further apart, there is a higher chance that only one is affected.

For the L1 band, the chirp length is about 1 microsecond ($1 \times 10^{-6}s$), with the speed of light, gives about 300m per chirp. Since 1 chirp is the maximum error in the bias, this 300m error is the upper bound from this source, but realistically, it will be fractions of chirps, so for about 0.25 chirps, the error is about 75m.

Bonus [1 points]

Suppose we already track the ideal peak before encountering multipath. If the secondary peak is sufficiently delayed from the ideal peak, it will no longer induce any error in our DLL tracking loop. How much multipath delay (in chips and meters) is sufficient for correlator spacings of $d = 0.1$, 0.5 , or 1 -chips? And, how much better is a smaller correlator spacing versus how much better is the GPS L1 P(Y) military signal?

Specifically, make a table that shows the minimum amount of delay in the reflected signal peak for which our early-minus-late correlator is no longer affected by multipath errors for the GPS L1 C/A versus L1 P(Y) signal. Make this table for $d = 0.1$ -chips, $d = 0.5$ -chips, and $d = 1$ -chip. Which is better?

Hint: There should be six data entries in this table, and you do not necessarily need to use any of the code you wrote previously in the problem (i.e., you can do a direct calculation of these entries).

ANSWER

```

[157]: ze = eval_ideal_autocorr(dt, 0.05)
        zl = eval_ideal_autocorr(dt, -0.05)
        ze_zl_01 = ze-zl

```

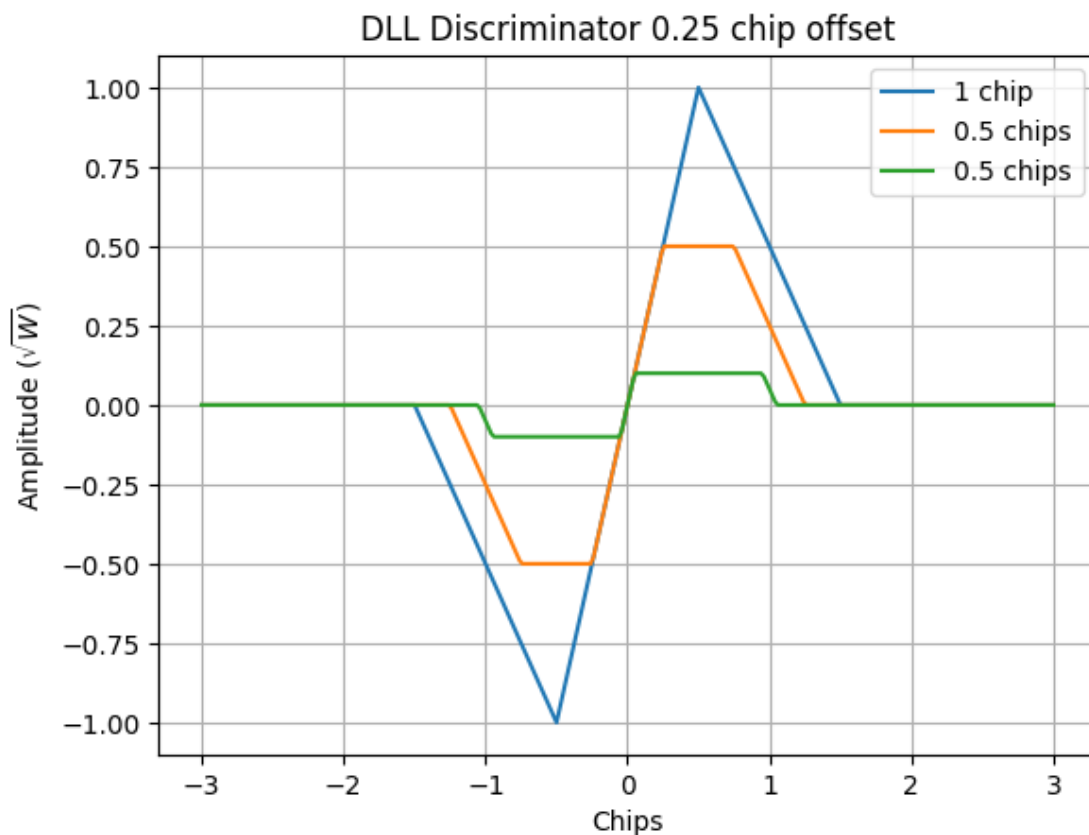
```

ze = eval_ideal_autocorr(dt, 0.25)
zl = eval_ideal_autocorr(dt, -0.25)
ze_zl_05 = ze-zl

ze = eval_ideal_autocorr(dt, 0.5)
zl = eval_ideal_autocorr(dt, -0.5)
ze_zl_1 = ze-zl

plt.xlabel("Chips")
plt.ylabel(r"Amplitude ( $\sqrt{W}$ )")
plt.title("DLL Discriminator 0.25 chip offset")
plt.plot(dt, ze_zl_1, label="1 chip")
plt.plot(dt, ze_zl_05, label="0.5 chips")
plt.plot(dt, ze_zl_01, label="0.5 chips")
plt.legend()
plt.grid()

```



[Compute the minimum delay in chips and meters for the C/A and P(Y) signals for the three different correlator spacings. Justify and discuss your results.]

The reflected peak has to be at least 1 chip away to avoid overlapping with the autocorrelation function peak in general. There also has to be an additional delay for each spacing d . For example, with a difference of 0.1 chip, the delay has to be an additional 0.05 chips to prevent it from affecting ZL. That is, the left side of the delayed peak will be outside of the $0.5d$ of ZL.

The pseudorange error is based off of the previously mentioned 300m for C/A signals, and therefore, because the military signal P(Y) is 10 times quicker, this causes its error to be about 30m

Correlator Spacing	Minimum delay [chips]	C/A signal [meters]	P(Y) signal [meters]
$d = 0.1$ chips	1.05	315	31.5
$d = 0.5$ chips	1.25	375	37.5
$d = 1$ chip	1.5	450	45

```
[158]: c_a_period = 1/1.023e6 #
m_per_chip = 3e8 * c_a_period

# TODO: Compute minimum delay in chips and meters for the C/A and P(Y) signals
# for the three different correlator spacings d=0.1, 0.5, 1 chips (in that
# order)
min_delay_chips = [1.05, 1.25, 1.5]
min_delay_meters_ca = [1.05*m_per_chip, 1.25*m_per_chip, 1.5*m_per_chip]
min_delay_meters_py = [1.05*m_per_chip/10, 1.25*m_per_chip/10, 1.5*m_per_chip/
10]
```

```
[159]: # *** DO NOT MODIFY THE CODE BELOW ***
print(
    "@P2.Bonus1\n",
    *np.round(min_delay_chips, 2),
    *np.round(min_delay_meters_ca, 2),
    *np.round(min_delay_meters_py, 2)
)
```

```
@P2.Bonus1
1.05 1.25 1.5 307.92 366.57 439.88 30.79 36.66 43.99
```

Bonus [1 point]

Though we saw an advantage to using the $d = 0.5$ -chip early-minus-late discriminator over the $d = 1$ -chip one, this benefit comes at a cost. Compare the two discriminator outputs that you obtained in parts (2) and (3). Explain one key drawback of using the narrower correlator. When may you want which correlator spacing?

Hint: You may find it helpful to review the lecture 11 slides on how the DLL tracking loop operates (or chapter 12 of the textbook) to answer this question.

ANSWER

With smaller chirp differences, if the signal error is very far off of the prompt, then there is a higher chance that both ZE and ZL are not on the main autocorrelation triangle at all. Then, because they are both equal and there is no perceived error, the GPS will not shift its prompt peak.

Narrower early-minus-late discriminators would be preferable in low noise environments.

Wider discriminators are ideal for noisy environments because they are more ideal for wider initial errors, and are more robust to noise in general.

Problem 3 (28 points): Extended Kalman Filter

Goal of this problem: In lecture 12, we learned about the conventional Kalman filter (KF) and the importance of Q and R . In this problem, students will learn to expand beyond the KF to account for nonlinearities in many real-world dynamical systems. Furthermore, students will see the importance of Q and R firsthand.

First, briefly revisit the assumptions for WLS from problem 2 on homework 2. For a system without dynamics (i.e., not moving), the Kalman Filters can be considered recursive least squares estimation. So, we inherit the assumptions from least squares but can now process measurements through time. As before, the error could be non-spherical or correlated. We would need to account for these in our measurement covariance R . The ideal weighting matrix W in weighted or generalized least squares (WLS and GLS) and the ideal measurement covariance matrix used in the Kalman Filter are related by $R = W^{-1}$. Intuitively, high variance means we want a lower weight.

However, most systems have dynamics. We must account for the system dynamics when processing measurements through time. Unlike least squares, the Kalman Filter leverages the dynamics[1] to improve our localization. However, vehicle dynamics are often *nonlinear*. The conventional Kalman filter (KF) cannot handle nonlinearity. So, we will leverage our insights from linearization that we used for Newton-Raphson to build up the *extended* Kalman filter (EKF)[2], which will allow us to have more general, *nonlinear* functions for our dynamics and measurement models.

Let us consider a simple 2D car scenario. Let the state of the vehicle μ_t be defined as:

$$\mu_t := [x_t \quad y_t \quad \theta_t]^\top,$$

where x_t is the x-position of the ground vehicle, y_t is its y-position, and θ_t is its orientation **in radians**. We will also consider a control vector u_t , which governs how our ground vehicle moves with time:

$$u_t := [s_t \quad \alpha_t]^\top$$

where s_t represents the vehicle's forward speed, in meters per second, and α_t represents its angular rate, **in radians per second**. Let our *noisy* state transition of the ground vehicle be represented as the following *nonlinear* function:

$$\begin{aligned} \mu_t &= f(\mu_{t-1}, u_t) + w_t, \\ &= \underbrace{\mu_{t-1} + \Delta t \begin{bmatrix} s_t \cos(\theta_{t-1}) \\ s_t \sin(\theta_{t-1}) \\ \alpha_t \end{bmatrix}}_{f(\mu_{t-1}, u_t)} + w_t, \end{aligned}$$

where f is the state transition model function (i.e., our dynamics model), Δt is the time in seconds between consecutive state estimates and w_t represents the state transition noise, which is assumed to be Gaussian such that $w_t \sim \mathcal{N}(0, Q)$, just like our assumptions in lecture. Note that, unlike the state μ_t and the state transition noise w_t , the control u_t is *not* a random variable. This is because we know what control we provide to the system, since we choose our vehicle's forward speed and angular rate.

For our measurements, we will consider ranging measurements from 4 stationary 2D beacons. Thus, we can represent our (*nonlinear*) ranging measurement[3] with respect to beacon i as:

$$h^{(i)}(\mu_t) = \sqrt{(x^{(i)} - x_t)^2 + (y^{(i)} - y_t)^2},$$

where $(x^{(i)}, y^{(i)})$ represents the known 2D position of the i^{th} stationary beacon. And we can thus represent our nonlinear measurement model as:

$$z_t = h(\mu_t) + v_t,$$

$$h(\mu_t) := [h^{(1)}(\mu_t) \quad h^{(2)}(\mu_t) \quad h^{(3)}(\mu_t) \quad h^{(4)}(\mu_t)]^\top,$$

where z_t represents our received measurement vector, $h(\mu_t)$ represents our expected measurement vector at state μ_t , and v_t represents our measurement noise, which is assumed to be Gaussian such that $v_t \sim \mathcal{N}(0, R)$, just like our assumptions in lecture. Note that our received measurement $z_t \in \mathbb{R}^4$, since we receive four ranging measurements (one from each stationary beacon). We can now write the EKF equations below.

Predict:

$$\hat{\mu}_{t|t-1} = f(\hat{\mu}_{t-1|t-1}, u_t), \quad \text{Predict the next state from dynamics}$$

$$P_{t|t-1} = F_t P_{t-1|t-1} F_t^\top + Q, \quad \text{Predict the next covariance from dynamics}$$

where F_t is the linearized state transition matrix, defined as $F_t := \left. \frac{\partial f}{\partial \mu} \right|_{\hat{\mu}_{t-1|t-1}, u_t}$. ### Update:

$$\tilde{y}_t = z_t - h(\hat{\mu}_{t|t-1}), \quad \text{Measured vs theoretical ranges}$$

$$K_t = P_{t|t-1} H_t^\top (R + H_t P_{t|t-1} H_t^\top)^{-1}, \quad \text{Measurement vs. dynamics weighting}$$

$$\hat{\mu}_{t|t} = \hat{\mu}_{t|t-1} + K_t \tilde{y}_t, \quad \text{Update the state from measurements}$$

$$P_{t|t} = (I - K_t H_t) P_{t|t-1}, \quad \text{Update the covariance from measurements}$$

where H_t is the linearized measurement matrix, defined as $H_t := \left. \frac{\partial h}{\partial \mu} \right|_{\hat{\mu}_{t|t-1}}$.

Indeed, the expressions of the EKF are very similar to the standard KF, except we use our nonlinear motion model and measurement model directly in equations (3) and (4), and we use linearizations of our motion model and measurement model for the F_t and H_t matrices.

3.1 Jacobain for Dynamics [2 points]

Find the Jacobian F_t for your EKF.

Hint: This Jacobian $F_t \in \mathbb{R}^{3 \times 3}$ will be defined as:

$$F_t := \left. \frac{\partial f}{\partial \mu} \right|_{\hat{\mu}_{t-1|t-1}, u_t},$$

$$= \left[\begin{array}{ccc} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} & \frac{\partial f_1}{\partial \theta} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} & \frac{\partial f_2}{\partial \theta} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial y} & \frac{\partial f_3}{\partial \theta} \end{array} \right] \bigg|_{\hat{\mu}_{t-1|t-1}, u_t}$$

Note, this notation indicates that, after taking the matrix derivative, you will evaluate F_t at $\mu = \hat{\mu}_{t-1|t-1}$ and $u = u_t$. Make sure to include the subscripts!

Note: You can check your work with Wolfram Alpha, Mathematica, or Matlab, but you must show your derivation.

ANSWER

$$\begin{aligned} f(\mu_{t-1}, u_t) &= \mu_{t-1} + \Delta t \begin{bmatrix} s_t \cos(\theta_{t-1}) \\ s_t \sin(\theta_{t-1}) \\ \alpha_t \end{bmatrix} \\ &= \begin{bmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{bmatrix} + \Delta t \begin{bmatrix} s_t \cos(\theta_{t-1}) \\ s_t \sin(\theta_{t-1}) \\ \alpha_t \end{bmatrix} \\ F_t &= \begin{bmatrix} 1 & 0 & -\Delta t s_t \sin(\theta_{t-1}) \\ 0 & 1 & \Delta t s_t \cos(\theta_{t-1}) \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

3.2 Jacobian for Measurement Function [2 points]

Find the Jacobian H_t for your EKF. Let $(x^{(i)}, y^{(i)})$ represent the position of the i^{th} beacon and consider that we have four beacons (and their corresponding ranging measurements) available at all times.

Hint: This Jacobian $H_t \in \mathbb{R}^{4 \times 3}$ and will be defined as:

$$\begin{aligned} H_t &:= \left. \frac{\partial h}{\partial \mu} \right|_{\mu = \hat{\mu}_{t|t-1}}, \\ &= \left. \begin{bmatrix} \frac{\partial h_1}{\partial x} & \frac{\partial h_1}{\partial y} & \frac{\partial h_1}{\partial \theta} \\ \vdots & \vdots & \vdots \\ \frac{\partial h_4}{\partial x} & \frac{\partial h_4}{\partial y} & \frac{\partial h_4}{\partial \theta} \end{bmatrix} \right|_{\hat{\mu}_{t|t-1}}. \end{aligned}$$

Note, after taking the derivative, you will evaluate H_t at $\mu = \hat{\mu}_{t|t-1}$ which is the *predicted* state (our state estimate after the predict step).

Note: You can check your work with Wolfram Alpha, Mathematica, or Matlab, but you must show your derivation.

ANSWER

$$h^{(i)}(\mu_t) = \sqrt{(x^{(i)} - x_t)^2 + (y^{(i)} - y_t)^2}$$

$$\begin{aligned}
H_t &= \begin{bmatrix} \frac{2(x^1-x_t)(-1)}{2\sqrt{(x^1-x_t)^2+(y^1-y_t)^2}} & \frac{2(y^1-y_t)(-1)}{2\sqrt{(x^1-x_t)^2+(y^1-y_t)^2}} & 0 \\ \vdots & \vdots & \vdots \\ \frac{2(x^4-x_t)(-1)}{2\sqrt{(x^4-x_t)^2+(y^4-y_t)^2}} & \frac{2(y^4-y_t)(-1)}{2\sqrt{(x^4-x_t)^2+(y^4-y_t)^2}} & 0 \end{bmatrix} \\
&= \begin{bmatrix} \frac{-(x^1-x_t)}{\sqrt{(x^1-x_t)^2+(y^1-y_t)^2}} & \frac{-(y^1-y_t)}{\sqrt{(x^1-x_t)^2+(y^1-y_t)^2}} & 0 \\ \vdots & \vdots & \vdots \\ \frac{-(x^4-x_t)}{\sqrt{(x^4-x_t)^2+(y^4-y_t)^2}} & \frac{-(y^4-y_t)}{\sqrt{(x^4-x_t)^2+(y^4-y_t)^2}} & 0 \end{bmatrix}
\end{aligned}$$

3.3 True State Dynamics [3 points]

Let us first simulate the *true* state dynamics μ_t . Create a function called `simulate_noisy_state_trans(mu_cur, u_cur, Q)`, which takes in the current true state `mu_cur`, control `u_cur`, process noise `Q`, and outputs the propagated state by following the nonlinear state propagation model defined in Eq. (1). Initialize the true initial state as $\mu_0 = [0 \ 0 \ 0]^\top$ and let the state process noise covariance $Q = \text{diag}([0.1, 0.1, \mathbf{0.01}])$ (Note that the process noise for the orientation is different!). Define the control vector u_t with a constant speed of $s_t = 5$ meters per second and a constant angular rate $\alpha_t = 0.1$ radians per second for all time steps t .

Plot the simulated true state trajectory, using the provided plot function. Also, check your implementation in autograder.

Hint 1: First create a function `state_trans_md1(mu_cur, u_cur)` which represents the state transition model function $f(\cdot)$ and does *not* inject noise. Then your `simulate_noisy_state_trans` function can call this function and inject noise according to the covariance matrix Q . This may also be helpful when you implement your EKF in part (7) since your EKF will need to evaluate $f(\cdot)$ in the predict step (see Eq. 3).

Hint 2: If a car is moving with no noise* at a constant speed of $s_t = 5$ meters per second and a constant angular rate $\alpha_t = 0.1$ radians per second, what shape would you expect it to make? What is the radius of this shape? Make sure you add different noise at each timestep!*

Note: A diagonal covariance matrix means the noise added to x_t , y_t , and θ_t are *uncorrelated*. For a joint Gaussian, this also means that the noise added is independent of each other. Moreover, the noise must be independent of any part x_t , y_t , or θ_t of the state μ_t . The measurements below have a similar theoretical basis.

ANSWER

[Submit code to autograder (2 points)]

[Provide 4 plots for different runs (1 point)]

```
[11]: # @title
import numpy as np
import matplotlib.pyplot as plt

# constants (do not change)
def setup_environment():
```

```

delta_t = 1 # time step
Q = np.diag(np.array([0.1, 0.1, 0.01])) # process noise covariance
R = 10 * np.eye(4) # measurement noise covariance
seed = 0 # random seed for reproducibility (do not change!!!)
lent = 100 # length of trajectory
n_mc = 4 # number of Monte Carlo simulations
mu0 = np.array([0, 0, 0]) # initial state [s_0, alpha, theta]

# control input
st = 5 # speed
alpha = 0.1 # angular rate
u_cur = np.array([st, alpha]) # current control input (2x1) [s_t, alpha]

# set random seed
np.random.seed(seed)

env = {
    'delta_t': delta_t,
    'Q': Q,
    'R': R,
    'lent': lent,
    'n_mc': n_mc,
    'mu0': mu0,
    'u_cur': u_cur,
    'seed': seed
}

return env

# nominal model
def state_trans_mdl(mu_cur, u_cur):
    """
    Nominal state transition model
    Inputs:
        mu_cur: current state mean (3,) [s_t, alpha, theta]
        u_cur: current control input (2,) [s_t, alpha]
    Outputs:
        mu_next: next state mean (3,) [s_t+1, alpha, theta]
    """
    # *****
    # State Transition Model *****
    # *****
    # TODO: replace with correct state transition model
    delta_t = 1

    s_t, alpha = u_cur

```

```

    mu_next = mu_cur + delta_t * np.array([s_t * np.cos(mu_cur[2]), s_t * np.
↪sin(mu_cur[2]), alpha])

    return mu_next

# *****
# *****

    return mu_next

# noisy dynamics
def simulate_noisy_state_trans(mu_cur, u_cur, Q):
    """
    Simulate noisy state transition
    Inputs:
        mu_cur: current state mean (3,) [s_t, alpha, theta]
        u_cur: current control input (2,) [s_t, alpha]
        Q: process noise covariance (3x3)
    Outputs:
        mu_next: next state mean (3x1) [s_t+1, alpha
    """
    noise = np.random.multivariate_normal(np.zeros(3), Q)

    # *****
    # State Transition Model with Noise *****
    # *****
    # TODO: implement the state transition with process noise (use noise above)
    mu_next = state_trans_mdl(mu_cur, u_cur) + noise

    # *****
    # *****

    return mu_next

# generate trajectories
def simulate_trajectories(env):
    """
    Simulate multiple trajectories with and without noise
    Outputs:
        mu_true: true trajectory with noise (3 x lent x n_mc)
        mu_nonoise: trajectory without noise (3 x lent)
    """
    lent = env['lent']
    n_mc = env['n_mc']
    u_cur = env['u_cur']

```

```

Q = env['Q']
mu0 = env['mu0']
seed = env['seed']

np.random.seed(seed)

mu_true = np.zeros((3, lent, n_mc))      # true trajectory with noise
mu_nonoise = np.zeros((3, lent))         # trajectory without noise

# simulate no-noise trajectory
for i in range(1, lent):
    mu_cur = mu_nonoise[:, i - 1]
    mu_next = state_trans_mdl(mu_cur, u_cur)
    mu_nonoise[:, i] = mu_next

# simulate noisy trajectory
for mci in range(4):
    mu_true[:, 0, mci] = mu0

    for i in range(1, lent):
        mu_cur = mu_true[:, i - 1, mci]
        mu_next = simulate_noisy_state_trans(mu_cur, u_cur, Q)
        mu_true[:, i, mci] = mu_next

return mu_true, mu_nonoise

def plot_trajectories(mu_true, mu_nonoise, n_mc):
    """
    Plot the simulated trajectories
    Inputs:
        mu_true: true trajectory with noise (3 x lent x n_mc)
        mu_nonoise: trajectory without noise (3 x lent)
        n_mc: number of Monte Carlo simulations
    Outputs:
        None
    """

    plt.subplots(2, 2, figsize=(8,8))

    for mci in range(n_mc):
        # plot the trajectory
        plt.subplot(2, 2, mci + 1)
        plt.plot(mu_true[0, :, mci], mu_true[1, :, mci], label='trajectory_0
↪(with noise)')

```

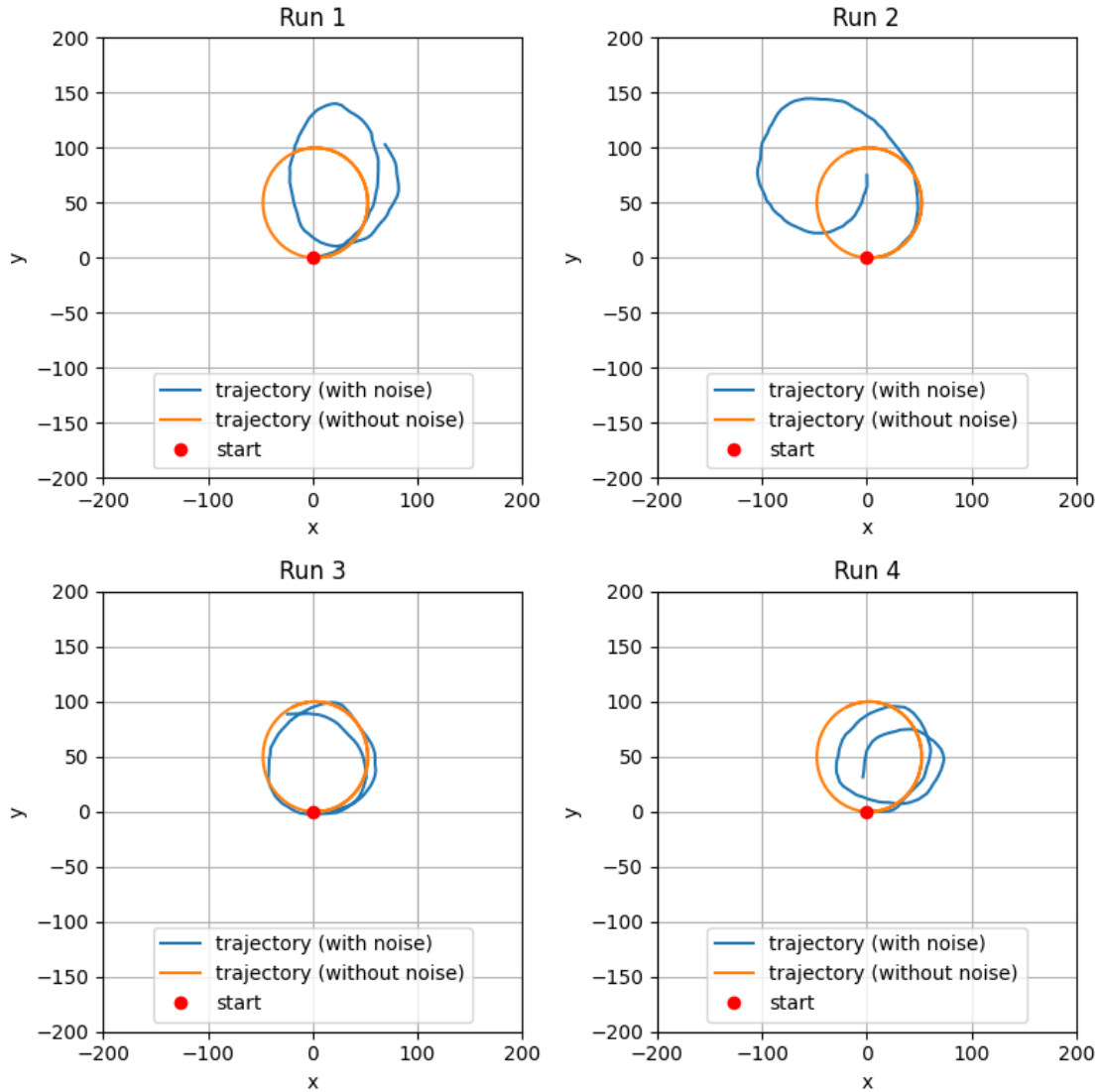
```

plt.plot(mu_nonoise[0, :], mu_nonoise[1, :], label='trajectory (without_
↳noise)')
plt.plot(mu_true[0, 0, mci], mu_true[1, 0, mci], 'ro', label='start')
plt.title('Run ' + str(mci + 1))
plt.xlabel('x')
plt.ylabel('y')
# plt.axis('equal')
plt.grid(True)
plt.xlim([-200, 200])
plt.ylim([-200, 200])
plt.legend(loc='lower center')

plt.tight_layout()
plt.show()

# simulate and plot trajectories
env = setup_environment()
mu_true, mu_nonoise = simulate_trajectories(env)
plot_trajectories(mu_true, mu_nonoise, env['n_mc'])

```



```
[12]: # *** DO NOT MODIFY THE CODE BELOW ***
sol_33 = np.hstack([mu_true.flatten(), mu_nonoise.flatten()])
print("@P3.3\n", *np.round(sol_33, 2))
```

@P3.3

```
0.0 0.0 0.0 0.0 5.13 4.87 5.12 5.29 10.62 10.35 9.82 10.91 15.47 15.44 14.91
16.71 20.31 19.82 19.41 21.43 24.65 24.28 23.8 25.95 29.01 28.01 28.04 29.77
32.37 31.76 33.15 33.09 36.76 35.25 37.92 36.17 40.34 38.76 41.87 39.94 44.53
42.04 45.78 43.44 47.43 44.88 49.15 47.51 50.06 46.74 52.48 51.46 52.67 47.32
55.39 54.51 54.29 48.09 57.59 56.27 57.06 48.9 59.36 57.4 58.58 48.71 60.15
59.08 59.59 48.52 59.46 60.24 60.45 47.97 59.53 61.02 61.7 47.63 59.31 60.55
62.09 46.54 58.06 58.77 62.24 45.88 56.14 57.45 62.48 44.58 53.55 55.49 62.5
42.37 50.65 53.39 62.56 39.53 47.4 50.96 61.44 36.61 44.51 47.55 59.75 33.51
```

41.5 44.46 58.15 30.3 38.75 40.13 56.76 27.38 35.48 35.97 56.08 23.74 31.88
 30.87 53.45 20.3 28.27 25.77 50.68 17.89 23.0 20.66 47.2 14.45 18.08 15.96 43.63
 10.75 13.43 11.64 39.8 7.42 8.29 7.05 35.76 2.83 3.26 2.54 31.34 -1.43 -1.75
 -1.71 27.44 -5.34 -6.0 -5.85 22.3 -10.04 -10.03 -9.59 17.56 -14.47 -14.03 -12.75
 12.92 -18.77 -18.22 -16.31 8.08 -23.05 -22.0 -19.88 3.69 -27.86 -25.36 -22.81
 0.35 -32.85 -29.05 -24.84 -3.25 -38.71 -32.64 -26.21 -5.98 -43.86 -35.29 -26.99
 -8.39 -48.77 -37.72 -28.54 -10.98 -53.51 -40.24 -28.95 -13.39 -58.47 -40.58
 -28.76 -14.81 -63.46 -41.56 -27.18 -16.91 -68.34 -41.75 -25.64 -17.79 -73.2
 -41.89 -22.23 -19.13 -77.37 -42.28 -18.15 -20.61 -81.31 -41.97 -13.79 -21.97
 -84.99 -40.44 -9.69 -21.7 -87.63 -38.29 -4.12 -21.89 -90.05 -35.61 1.06 -21.93
 -92.61 -32.29 6.0 -20.84 -94.69 -28.41 10.81 -19.64 -97.38 -24.27 15.79 -18.7
 -99.55 -20.23 20.91 -17.57 -100.61 -16.19 25.89 -15.63 -100.97 -11.95 31.17
 -14.16 -101.34 -6.9 35.88 -12.99 -102.62 -1.75 40.67 -11.19 -103.93 3.37 45.22
 -8.12 -104.19 8.13 49.58 -4.17 -103.23 13.51 53.55 -0.08 -101.85 18.49 57.36
 3.65 -99.46 23.19 60.52 8.21 -97.25 27.83 63.26 12.97 -95.51 32.51 66.34 17.58
 -91.98 36.16 68.74 22.65 -89.24 40.29 71.06 27.26 -85.5 43.67 72.67 32.25 -81.72
 46.28 73.84 37.19 -78.12 48.29 72.96 41.86 -73.59 49.64 71.08 46.48 -68.56 51.13
 67.89 51.28 -63.66 50.75 64.46 55.32 -59.12 49.89 59.84 58.52 -54.35 48.83 56.16
 61.42 -49.81 47.58 51.7 65.01 -44.95 45.45 47.51 68.36 -40.77 42.71 42.29 71.28
 -35.79 39.34 37.54 73.35 -31.08 35.24 32.5 76.01 -27.08 31.58 27.68 78.38 -22.76
 27.84 22.97 80.57 -18.95 24.41 18.29 81.9 -14.88 20.28 13.35 81.91 -11.32 15.54
 9.33 81.12 -8.92 10.5 5.61 80.2 -6.19 5.41 2.82 79.8 -4.27 0.68 0.09 78.92 -3.01
 -4.54 -0.89 76.62 -1.05 -9.12 -1.92 74.1 0.39 -14.18 -2.24 71.75 0.23 -18.89
 -2.8 69.05 0.2 -24.14 -3.6 0.0 0.0 0.0 0.56 0.18 0.11 0.15 2.25 0.45 0.31
 0.11 3.54 1.36 0.49 0.11 5.1 2.31 1.27 0.42 7.89 4.43 2.14 1.93 11.14 7.12 3.46
 4.49 14.68 10.44 4.95 8.39 17.73 13.97 6.79 12.06 21.39 17.36 9.52 15.62 24.68
 20.68 12.43 19.25 28.88 25.34 15.61 23.07 32.45 30.05 19.85 26.92 37.31 34.47
 23.67 30.89 41.81 38.88 27.97 36.04 45.79 44.38 32.39 40.45 50.27 49.16 37.75
 45.0 54.64 54.07 43.06 49.72 59.66 59.21 47.96 54.87 64.52 63.69 52.72 59.92
 69.58 68.4 56.8 64.77 74.36 73.22 61.43 69.53 78.82 78.27 65.35 74.26 83.3 82.87
 69.39 78.12 88.31 87.01 73.34 82.19 93.62 90.64 77.14 86.26 98.2 94.9 81.38
 89.74 102.83 98.7 85.12 92.83 107.87 102.48 89.26 94.87 113.18 106.66 93.05
 95.29 117.56 110.73 96.6 95.69 121.32 114.32 98.81 95.41 125.1 118.19 99.25
 93.87 127.91 121.14 98.64 92.56 130.37 124.79 97.6 90.84 132.45 127.43 95.99
 88.96 135.62 130.1 94.28 86.32 138.67 132.97 92.07 83.68 140.07 135.2 90.23
 80.26 139.65 137.95 87.71 76.6 138.31 140.65 84.89 73.15 136.24 142.21 82.01
 70.24 133.91 142.69 78.48 65.74 130.67 143.45 74.81 60.68 127.09 143.93 70.75
 56.26 122.7 143.99 66.49 51.09 118.64 144.44 62.22 45.9 114.3 144.7 57.57 40.74
 110.08 144.61 51.88 35.69 106.14 143.6 46.89 31.19 101.79 142.65 41.59 26.78
 96.97 140.42 36.36 23.05 91.88 137.76 31.59 20.44 87.37 134.91 26.9 17.78 82.38
 130.75 22.68 15.15 77.69 126.39 18.49 13.46 72.96 121.91 13.87 11.56 68.33
 118.21 10.36 10.22 63.5 112.97 6.89 9.04 58.27 108.54 4.77 8.45 53.17 104.75
 2.29 8.34 48.31 99.67 0.21 7.67 42.92 94.96 -1.4 7.26 37.79 89.87 -2.0 7.29
 33.54 85.56 -2.69 8.19 29.02 80.73 -2.22 9.67 24.84 76.13 -2.1 11.22 21.98 71.24
 -1.79 13.87 18.59 66.12 -1.4 16.29 16.14 61.55 -0.9 20.32 13.67 57.05 1.14 24.74
 12.15 52.02 3.52 29.01 11.04 48.37 6.25 33.17 10.37 43.72 9.17 37.95 11.31 40.22
 12.78 42.78 12.43 37.58 17.12 47.29 13.46 33.89 21.7 52.14 14.88 30.96 26.19
 56.62 16.12 28.64 31.1 60.29 17.82 26.63 36.53 63.78 20.29 24.58 41.77 66.71

23.82 23.23 46.61 69.79 27.73 22.44 51.31 72.18 31.89 22.44 55.96 74.09 35.29
 22.57 60.72 74.97 39.32 23.25 64.81 74.69 43.98 24.97 68.59 74.37 47.82 26.94
 72.29 72.95 51.96 28.56 75.98 71.92 55.87 31.85 79.44 70.66 60.69 34.62 82.31
 68.89 65.74 38.33 84.72 66.81 70.83 42.49 86.76 63.86 75.9 47.03 88.15 59.88
 81.54 51.23 88.55 55.54 86.13 56.22 89.19 50.74 90.92 60.56 88.8 46.0 95.19
 65.18 88.99 41.4 98.98 69.99 88.76 36.32 102.89 75.13 88.66 31.35 0.0 0.0 0.0
 0.0 0.2 0.14 0.05 0.08 0.2 0.23 0.14 0.1 0.29 0.28 0.14 0.16 0.54 0.45 0.18 0.29
 0.68 0.66 0.29 0.58 0.76 0.72 0.26 0.77 0.6 0.79 0.35 0.73 0.63 0.82 0.48 0.71
 0.73 0.83 0.65 0.78 0.98 0.91 0.72 0.75 0.99 1.11 0.84 0.87 1.11 1.28 0.94 0.95
 1.17 1.44 1.08 1.19 1.13 1.48 1.23 1.32 1.18 1.53 1.41 1.34 1.35 1.59 1.64 1.36
 1.36 1.7 1.62 1.59 1.35 1.73 1.71 1.68 1.45 1.76 1.84 1.93 1.52 1.67 2.0 1.93
 1.54 1.83 2.09 1.99 1.6 2.03 2.09 2.18 1.6 2.14 2.24 2.15 1.72 2.22 2.27 2.28
 1.86 2.22 2.17 2.37 1.9 2.32 2.09 2.54 1.88 2.27 2.21 2.74 1.83 2.2 2.41 3.04
 2.05 2.19 2.45 3.05 2.25 2.2 2.64 3.19 2.37 2.31 2.95 3.39 2.54 2.38 3.29 3.45
 2.66 2.4 3.24 3.52 2.62 2.65 3.45 3.54 2.61 2.64 3.57 3.68 2.63 2.56 3.54 3.67
 2.92 2.66 3.67 3.86 3.21 2.62 3.76 3.98 3.4 2.7 3.79 3.9 3.6 2.83 3.86 3.9 3.72
 3.11 3.88 4.16 3.81 3.0 3.93 4.28 3.98 3.11 4.05 4.39 4.13 3.15 4.06 4.45 4.17
 3.11 4.24 4.47 4.25 3.11 4.26 4.68 4.2 3.21 4.61 4.79 4.36 3.27 4.58 4.97 4.37
 3.39 4.56 5.15 4.42 3.54 4.65 5.41 4.45 3.62 4.75 5.62 4.48 3.77 4.88 5.77 4.47
 4.07 5.03 5.78 4.76 4.19 5.21 5.87 4.74 4.2 5.31 5.98 4.68 4.24 5.44 6.05 4.88
 4.27 5.61 6.03 4.91 4.16 5.71 6.14 4.93 4.19 5.76 6.24 5.03 4.42 5.88 6.24 5.07
 4.57 5.98 6.2 5.01 4.58 6.07 6.36 5.03 4.42 6.23 6.49 5.16 4.51 6.34 6.65 5.31
 4.58 6.31 6.67 5.5 4.95 6.4 6.84 5.56 5.0 6.42 6.91 5.72 5.13 6.41 7.23 5.82
 5.21 6.65 7.32 5.91 5.15 6.71 7.31 6.09 5.38 6.86 7.33 6.15 5.35 6.95 7.46 6.48
 5.46 7.15 7.61 6.54 5.59 7.43 7.78 6.49 5.6 7.44 8.06 6.61 5.72 7.54 8.29 6.57
 5.97 7.57 8.5 6.71 5.84 7.9 8.69 6.96 5.95 8.03 8.78 7.13 6.02 8.12 8.76 7.21
 6.11 8.14 8.92 7.15 6.19 8.27 9.04 7.16 6.24 8.47 9.15 7.21 6.46 8.55 9.4 7.38
 6.57 8.57 9.48 7.34 6.6 8.56 9.7 7.32 6.7 8.55 9.65 7.41 7.01 8.61 9.77 7.57
 6.93 8.83 9.9 7.67 7.06 8.94 9.96 7.97 7.22 9.03 10.12 8.03 7.32 9.21 10.39 7.93
 7.38 9.29 10.5 8.13 7.47 9.28 10.68 8.26 7.5 9.39 10.77 8.48 7.55 9.39 10.81
 8.52 7.69 9.42 10.91 8.51 7.85 9.46 10.82 8.62 7.84 9.5 11.02 0.0 5.0 9.98 14.88
 19.65 24.26 28.65 32.77 36.6 40.08 43.19 45.89 48.16 49.97 51.31 52.16 52.51
 52.36 51.72 50.58 48.97 46.89 44.36 41.42 38.09 34.4 30.4 26.11 21.59 16.88
 12.03 7.08 2.08 -2.91 -7.85 -12.68 -17.37 -21.85 -26.09 -30.04 -33.67 -36.94
 -39.82 -42.27 -44.27 -45.81 -46.86 -47.42 -47.49 -47.05 -46.12 -44.7 -42.81
 -40.46 -37.69 -34.52 -30.98 -27.1 -22.92 -18.5 -13.86 -9.06 -4.14 0.84 5.84
 10.81 15.69 20.44 25.01 29.36 33.44 37.21 40.63 43.67 46.3 48.49 50.23 51.48
 52.25 52.52 52.29 51.56 50.35 48.65 46.49 43.9 40.89 37.49 33.75 29.69 25.37
 20.81 16.07 11.2 6.24 1.24 -3.75 -8.67 -13.48 -18.13 0.0 0.0 0.5 1.49 2.97 4.92
 7.31 10.14 13.36 16.95 20.86 25.07 29.53 34.19 39.0 43.93 48.92 53.92 58.87
 63.74 68.48 73.02 77.34 81.38 85.11 88.49 91.48 94.06 96.19 97.87 99.06 99.77
 99.98 99.69 98.9 97.62 95.87 93.65 91.0 87.94 84.51 80.72 76.63 72.27 67.69
 62.93 58.05 53.08 48.08 43.1 38.18 33.39 28.76 24.34 20.18 16.32 12.79 9.63 6.88
 4.56 2.69 1.29 0.38 -0.03 0.05 0.63 1.71 3.27 5.29 7.76 10.65 13.94 17.58 21.55
 25.8 30.3 34.99 39.83 44.77 49.76 54.75 59.7 64.55 69.25 73.76 78.04 82.03 85.7
 89.02 91.94 94.45 96.51 98.1 99.22 99.84 99.96 99.59 98.72 97.36 95.52 0.0 0.1
 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2.0 2.1
 2.2 2.3 2.4 2.5 2.6 2.7 2.8 2.9 3.0 3.1 3.2 3.3 3.4 3.5 3.6 3.7 3.8 3.9 4.0 4.1

4.2 4.3 4.4 4.5 4.6 4.7 4.8 4.9 5.0 5.1 5.2 5.3 5.4 5.5 5.6 5.7 5.8 5.9 6.0 6.1
 6.2 6.3 6.4 6.5 6.6 6.7 6.8 6.9 7.0 7.1 7.2 7.3 7.4 7.5 7.6 7.7 7.8 7.9 8.0 8.1
 8.2 8.3 8.4 8.5 8.6 8.7 8.8 8.9 9.0 9.1 9.2 9.3 9.4 9.5 9.6 9.7 9.8 9.9

3.4 Simulating True Measurements [3 points]

We also want to simulate our measurements with respect to the *true state*. Create a function called `simulate_noisy_meas(mu_cur, R)` which takes in the current true state `mu_cur` and measurement noise covariance `R` and outputs the simulated noisy measurement vector by following the nonlinear measurement model defined in Eq. (2). Simulate your measurements with the following beacon positions:

$$\begin{aligned}(x^{(1)}, y^{(1)}) &= (-120, +120), \\(x^{(2)}, y^{(2)}) &= (+120, +120), \\(x^{(3)}, y^{(3)}) &= (+120, -120), \\(x^{(4)}, y^{(4)}) &= (-120, -120).\end{aligned}$$

These beacon placements will ensure good geometry (i.e., think about DOP) through our trajectory. Let the measurement noise covariance $R = 10I_4$, where I_4 is the 4×4 identity matrix. Hence, the measurement errors are uncorrelated, spherical errors.

Plot the noisy measurements for a stationary vehicle at $(x, y) = (-50, +100)$ for 100 time steps for all four beacons as a function of time. The x-axis is the time step, and the y-axis is the ranging measurement. Also, submit your code to autograder.

Hint: First create a function `meas_md1(mu_cur)` which represents the measurement model function $h(\cdot)$ and does *not* inject noise. Then your `simulate_noisy_meas` function can call this function and inject measurement noise according to the covariance matrix R . This may also be helpful when you implement your EKF in part (7) since your EKF will need to evaluate $h(\cdot)$ in the update step (see Eq. 4).

ANSWER

[Submit your code to autograder (2 points)]

[Provide a time series plot of the noisy measurements (1 point)]

```
[13]: # @title
def meas_md1(mu_cur):
    """
    Measurement model
    Inputs:
        mu_cur: current state mean (3,) [x_t, y_t, theta]
    Outputs:
        z: measurement vector (4,) [d_1, d_2, d_3, d_4]
    """
    posb = 120
    # beacon positions
    xy_beacon = np.array([[-posb, posb], [posb, posb], [posb, -posb], [-posb,
    ↪ -posb]])
```

```

# *****
# Measurement Model *****
# *****
# TODO: replace with correct state transition model
z = np.zeros(4)
x,y,_ = mu_cur

for i, b in enumerate(xy_beacon):
    z[i] = np.sqrt( (x - b[0])**2 + (y - b[1])**2 )

# *****
# *****

return z

# simulate noisy meas
def simulate_noisy_meas(mu_cur, R):
    """
    Simulate noisy measurement
    Inputs:
        mu_cur: current state mean (3,) [s_t, alpha, theta]
        R: measurement noise covariance (4x4)
    Outputs:
        z: measurement vector (4,) [d_1, d_2, d_3, d_4]
    """
    noise = np.random.multivariate_normal(np.zeros(4), R)

    # *****
    # State Transition Model with Noise *****
    # *****
    # TODO: implement the measurement model with measurement noise (use noise_
    ↪ above)
    z = meas_md1(mu_cur) + noise

    # *****
    # *****

    return z

def simulate_measurements(pos, R, sim_num, seed=0):
    sim_num = 100
    z = np.zeros((4, sim_num))
    np.random.seed(seed)

    for i in range(sim_num):
        z[:, i] = simulate_noisy_meas(pos, R)

    return z

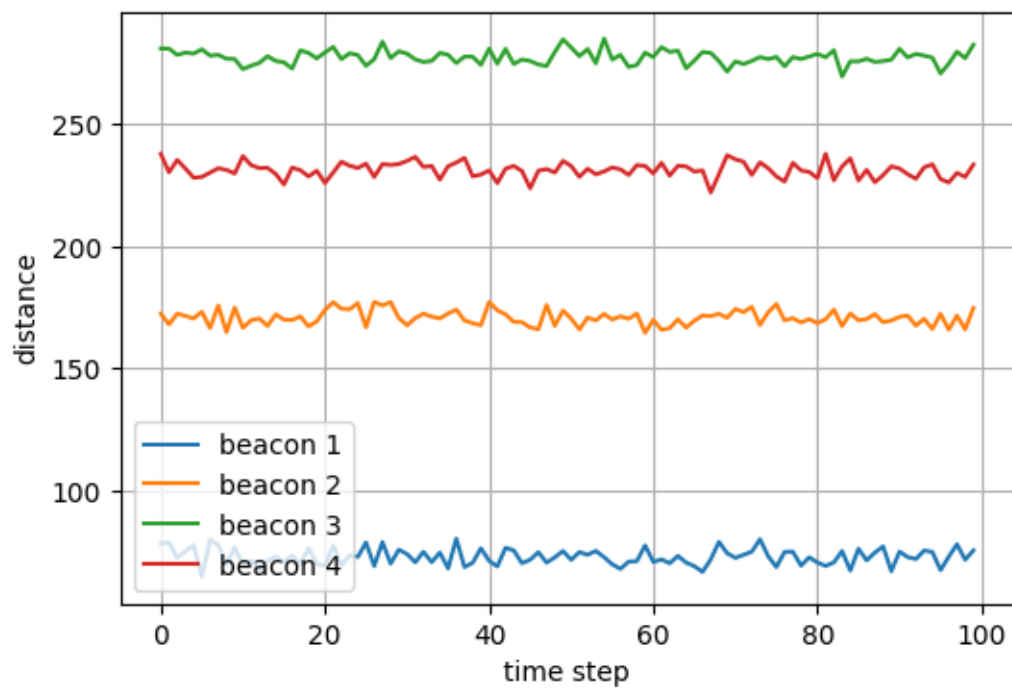
```

```

# plot the four measurements
def plot_measurements(z):
    sim_num = z.shape[1]
    plt.subplots(1, 1, figsize=(6, 4))
    plt.plot(np.arange(sim_num), z[0, :], label='beacon 1')
    plt.plot(np.arange(sim_num), z[1, :], label='beacon 2')
    plt.plot(np.arange(sim_num), z[2, :], label='beacon 3')
    plt.plot(np.arange(sim_num), z[3, :], label='beacon 4')
    plt.xlabel('time step')
    plt.ylabel('distance')
    plt.grid(True)
    plt.legend()
    plt.show()

# simulate noisy meas
pos = np.array([-50, 100, 0])
z = simulate_measurements(pos, env['R'], sim_num=100)
plot_measurements(z)

```



```

[14]: # *** DO NOT MODIFY THE CODE BELOW ***
sol_34 = z.flatten()
print("@P3.4\n", *np.round(sol_34, 2))

```

@P3.4

78.38 78.71 72.47 75.21 77.53 64.73 79.98 77.65 69.99 76.69 69.49 71.19 67.7
71.19 73.01 70.67 73.36 69.93 76.4 70.05 69.12 77.51 69.42 73.46 72.83 78.76
69.09 78.88 70.08 75.8 73.99 70.6 74.93 70.67 74.62 68.09 80.34 68.64 70.45
76.36 71.23 68.93 76.56 75.51 70.26 71.68 74.78 70.46 72.68 75.24 71.63 74.83
73.69 75.29 72.75 69.78 67.93 70.91 71.08 77.53 70.79 71.82 70.23 73.25 70.51
69.04 66.68 71.82 78.99 74.36 72.45 73.71 74.98 80.09 73.11 68.67 74.91 75.0
69.24 72.67 70.53 69.14 70.57 75.25 67.25 76.18 71.41 74.63 77.12 66.97 75.01
72.75 71.95 75.46 74.84 67.43 72.92 78.01 71.68 75.51 172.44 168.08 172.47
171.56 170.52 173.24 166.57 175.82 164.91 174.97 166.68 169.79 170.5 167.44
172.13 170.04 169.9 171.34 167.27 169.34 174.02 177.17 174.51 174.26 176.82
166.91 177.32 175.85 177.21 170.68 167.7 170.7 172.46 171.27 170.51 172.56
174.16 169.71 168.56 167.76 177.27 173.84 172.17 169.11 168.99 166.82 166.11
176.05 167.48 173.78 170.42 166.06 170.86 169.7 172.37 170.08 171.37 170.42
172.49 164.63 169.92 165.87 166.54 170.16 166.8 169.57 171.77 171.48 172.41
170.81 174.4 173.1 175.29 167.82 173.02 176.42 169.72 170.67 168.86 170.27 168.6
170.18 174.16 167.43 172.6 169.74 170.29 172.28 168.99 169.66 171.18 171.68
167.61 170.38 166.21 172.4 165.93 171.69 166.06 174.78 281.12 281.03 278.48
279.43 279.02 280.76 278.17 278.52 276.93 276.8 272.63 274.07 275.2 277.94
276.02 275.46 272.87 280.33 279.3 277.04 279.5 281.76 276.75 279.16 278.43
274.01 276.72 283.93 277.18 279.97 278.97 276.65 275.59 276.02 279.28 278.56
275.14 277.81 277.72 274.4 281.03 274.87 280.94 274.76 276.59 275.99 274.54
273.94 279.68 284.87 281.51 277.95 280.91 275.04 285.17 276.56 278.52 273.53
274.37 279.38 277.61 281.67 279.68 280.22 273.02 276.14 279.69 279.29 275.97
271.61 275.84 274.72 276.04 277.6 276.77 277.66 273.81 277.61 276.81 277.83
278.9 277.53 280.39 269.62 275.87 275.85 276.88 275.61 275.97 276.51 280.98
277.43 278.92 278.19 277.37 270.9 274.91 279.82 277.11 282.67 237.95 230.39
235.47 231.92 228.17 228.52 230.28 232.06 231.36 229.91 237.04 233.33 232.09
232.22 229.72 225.41 232.33 231.28 228.7 231.05 226.01 230.3 234.73 233.1 232.14
233.93 228.5 233.73 233.41 233.78 235.06 236.72 232.57 233.01 227.41 232.88
234.4 236.29 228.77 229.48 231.14 225.98 231.88 233.02 230.92 223.84 231.03
231.71 230.33 235.09 232.94 228.53 231.87 229.57 230.73 232.39 231.6 229.31
233.34 233.01 229.93 234.28 229.05 233.06 232.8 230.7 231.15 222.1 229.63 237.4
235.73 234.7 229.35 234.46 232.04 228.72 226.61 234.28 231.17 230.53 228.05
238.0 227.11 232.79 236.12 227.03 231.36 226.32 229.22 232.83 231.94 229.62
227.73 232.43 233.65 227.63 226.21 230.16 228.46 233.56

3.5 Implementing a State Transition Function [1 points]

Write a function called `get_F(mu_cur, u_cur)` which returns the linearized state transition matrix F_t given the current state estimate `mu_cur` (numpy array of size (3,)) and control `u_cur` (numpy array of size (2,)). Check that it passes the autograder

ANSWER

[Submit your code to autograder]

```
[15]: def get_F(mu_cur, u_cur):
        """
        Compute the Jacobian of the state transition model
```

```

Inputs:
    mu_cur: current state mean (3,) [s_t, alpha, theta]
    u_cur: current control input (2,) [s_t, alpha]
Outputs:
    F: Jacobian matrix (3x3)
"""

# *****
# Jacobian Computation *****
# *****
# TODO: replace with correct Jacobian computation
# F = np.zeros((3, 3)) # placeholder
F = np.array([
    [1, 0, -u_cur[0] * np.sin(mu_cur[2])],
    [0, 1, u_cur[0] * np.cos(mu_cur[2])],
    [0, 0, 1]
])
# *****
# *****

return F

```

```

[16]: # *** DO NOT MODIFY THE CODE BELOW ***
# For test (do not modify)
mu_test1 = np.array([10, 30, np.pi/4])
u_test1 = np.array([5, 0.1])
F_test1 = get_F(mu_test1, u_test1)

mu_test2 = np.array([20, -10, np.pi/6])
u_test2 = np.array([3, -0.2])
F_test2 = get_F(mu_test2, u_test2)

# for autograder
sol_35 = np.hstack([F_test1.flatten(), F_test2.flatten()])
print("@P3.5\n", *np.round(sol_35, 2))

```

@P3.5

1.0 0.0 -3.54 0.0 1.0 3.54 0.0 0.0 1.0 1.0 0.0 -1.5 0.0 1.0 2.6 0.0 0.0 1.0

3.6 Implementing Measurement Matrix Function [1 points]

Write a function called `get_H(mu_cur)` which returns the linearized measurement matrix H_t given the current (predicted) state estimate `mu_cur` (numpy array of (3,)). Check your function in autograder.

ANSWER

[Submit your code to autograder]

```

[17]: # @title
def get_H(mu_cur):
    """
    Compute the Jacobian of the measurement model
    Inputs:
        mu_cur: current state mean (3,) [s_t, alpha, theta]
    Outputs:
        H: Jacobian matrix (4x3)
    """
    # location of beacons
    xy_beacon = np.array([[-120, 120], [120, 120], [120, -120], [-120, -120]])

    # *****
    # Jacobian Computation *****
    # *****
    # TODO: replace with correct Jacobian computation

    x,y,_ = mu_cur

    d1 = np.sqrt((x - xy_beacon[0][0])**2 + (y - xy_beacon[0][1])**2)
    d2 = np.sqrt((x - xy_beacon[1][0])**2 + (y - xy_beacon[1][1])**2)
    d3 = np.sqrt((x - xy_beacon[2][0])**2 + (y - xy_beacon[2][1])**2)
    d4 = np.sqrt((x - xy_beacon[3][0])**2 + (y - xy_beacon[3][1])**2)

    H = np.array([
        [(x - xy_beacon[0][0]) / d1, (y - xy_beacon[0][1]) / d1, 0],
        [(x - xy_beacon[1][0]) / d2, (y - xy_beacon[1][1]) / d2, 0],
        [(x - xy_beacon[2][0]) / d3, (y - xy_beacon[2][1]) / d3, 0],
        [(x - xy_beacon[3][0]) / d4, (y - xy_beacon[3][1]) / d4, 0],
    ])

    # *****
    # *****

    return H

```

```

[18]: # *** DO NOT MODIFY THE CODE BELOW ***
# For test (do not modify)
mu_test1 = np.array([10, 30, np.pi/4])
H_test1 = get_H(mu_test1)
mu_test2 = np.array([20, -10, np.pi/6])
H_test2 = get_H(mu_test2)
# for autograder
sol_36 = np.hstack([H_test1.flatten(), H_test2.flatten()])

print("@P3.6\n", *np.round(sol_36, 2))

```

@P3.6

```
0.82 -0.57 0.0 -0.77 -0.63 0.0 -0.59 0.81 0.0 0.65 0.76 0.0 0.73 -0.68 0.0
-0.61 -0.79 0.0 -0.67 0.74 0.0 0.79 0.62 0.0
```

3.7 Implementing Kalman Filter [8 points]

Implement the extended Kalman filter. Start with an initial state covariance of $P_0 = \text{diag}([25, 25, 1])$, and sample your initial state estimate $\hat{\mu}_0$ with respect to the true initial state μ_0 and this initial covariance P_0 . In other words, $\hat{\mu}_0 \sim \mathcal{N}(\mu_0, P_0)$. Similar to part (3), use a Δt of 1 second. For your filter, use the same Q and R matrices from the previous subparts.

Plot the estimated state trajectory with the true state trajectory for 100 time steps. Clearly mark the estimated initial point, the true initial point, and the beacon locations. Ensure the x-axis corresponds to the x-position and the y-axis corresponds to the y-position of the state. For this problem, you only need to run one simulation case.

Also, include the implemented code in your writeup.

Hints: 1. Make sure you interpret the state orientation θ_t to be in **radians** (and, correspondingly, the angular rate α_t will be in radians per second). 2. You will have to re-compute F_t at every predict step since this matrix is linearized with respect to the state estimate $\mu_{t-1|t-1}$ and control u_t . Similarly, you will have to re-compute H_t at every update step since this matrix is linearized with respect to the predicted state estimate $\mu_{t|t-1}$. See the definitions in the problem statement and parts (1) and (2) for more details. This is akin to how we recalculated the geometry matrix G at each step when solving the navigation equation with Newton-Raphson (problem 1 on homework 2). 3. Your simulated measurements at each time step have to be taken with respect to the *true* state (not the estimated state). You can use your `simulate_noisy_meas` function from part (4) for this. 4. You may find it helpful to write the EKF predict and the EKF update steps in separate functions, e.g., `ekf_predict` and `ekf_update`. These functions can call the `get_F` and `get_H` functions you wrote in parts (5) and (6). After implementing these functions, you can continuously call the predict and update functions in your main for-loop to run your filter for 100 time steps. 5. You may find it helpful to first generate the *true* trajectory as in part 3. Second, generate the simulated measurements z_t from the *true trajectory*. Third, run the EKF using z_t in the update step.

Note: In a real system, you could initialize from WLS with Newton-Raphson. But, notice that the EKF will start tracking correctly within 2 timesteps, so the WLS initialization is not strictly necessary.

ANSWER

[Submit your code to autograder (6 points)]

[Provide a plot of the estimated state trajectory overlayed on the true state trajectory based on the specifications in the problem (2 points)]

```
[19]: def ekf_predict(mu_hat, u_cur, P_hat, Q_ekf):
      """
      EKF Prediction step
      Inputs:
          mu_hat: current state estimate (3,)
```

```

        u_cur: current control input (2,)
        P_hat: current covariance estimate (3x3)
        Q_ekf: process noise covariance (3x3)
    Outputs:
        mu_bar: predicted state estimate (3,)
        P_bar: predicted covariance estimate (3x3)
    """

    # *****
    # EKF Prediction Step *****
    # *****
    # TODO: implement the EKF prediction step
    # *****
    # *****

    F = get_F(mu_hat, u_cur)
    mu_bar = state_trans_mdl(mu_hat, u_cur)

    P_bar = F @ P_hat @ F.T + Q_ekf

    return mu_bar, P_bar

def ekf_update(mu_bar, z_cur, P_bar, R_ekf):
    """
    EKF Update step
    Inputs:
        mu_bar: predicted state estimate (3,)
        z_cur: current measurement (4,)
        P_bar: predicted covariance estimate (3x3)
        R_ekf: measurement noise covariance (4x4)
    Outputs:
        mu_hat: updated state estimate (3,)
        P_hat: updated covariance estimate (3x3)
    """

    # *****
    # EKF Update Step *****
    # *****
    # TODO: implement the EKF update step
    # *****
    # *****

    # Innovation
    y_err = z_cur - meas_mdl(mu_bar)

    # Kalman Gain
    H = get_H(mu_bar)
    K = P_bar @ H.T @ np.linalg.inv(R_ekf + H @ P_bar @ H.T)

```

```

    # Updated state and cov.
    mu_hat = mu_bar + K @ y_err
    P_hat = (np.eye(3) - K @ H) @ P_bar

    return mu_hat, P_hat

def run_ekf(Q_ekf, R_ekf, u_cur, mu_true, z_true, P0):
    """
    Run EKF over the entire trajectory

    Inputs:
        Q_ekf: process noise covariance (3x3)
        R_ekf: measurement noise covariance (4x4)
        u_cur: current control input (2,)
        mu_true: true trajectory (3 x lent)
        z_true: measurements (4 x lent)
        P0: initial covariance (3x3)

    Outputs:
        mu_hat: estimated trajectory (3 x lent)
        P_hat: estimated covariance (3 x 3 x lent)
    """
    lent = mu_true.shape[1]
    mu_hat = np.zeros((3, lent))
    P_hat = np.zeros((3, 3, lent))

    # initial estimate
    mu_hat[:, 0] = mu_true[:, 0] + np.random.multivariate_normal(np.zeros(3), P0)
    P_hat[:, :, 0] = P0

    for i in range(1, lent):
        # *****
        # EKF over entire trajectory *****
        # *****
        # TODO: implement the EKF over the entire trajectory
        mu_cur = mu_hat[:, i-1]
        P_cur = P_hat[:, :, i-1]

        mu_pre, P_pre = ekf_predict(mu_cur, u_cur, P_cur, Q_ekf)
        mu_post, P_post = ekf_update(mu_pre, z_true[:, i], P_pre, R_ekf)

        # mu_hat[:, i], P_hat[:, :, i] = ekf_predict

        mu_hat[:, i] = mu_post # placeholder
        P_hat[:, :, i] = P_post # placeholder
        # *****

```

```

# *****

return mu_hat, P_hat

def sim_true_traj_meas(mu0, u_cur, Q, R, lent, seed):
    """
    Simulate true (noisy) trajectory and measurements
    Inputs:
        mu0: initial state (3,)
        u_cur: current control input (2,)
        Q: process noise covariance (3x3)
        R: measurement noise covariance (4x4)
        lent: length of trajectory
    Outputs:
        mu_true: true trajectory (3 x lent)
        z_true: measurements (4 x lent)
    """
    mu_true = np.zeros((3, lent))
    z_true = np.zeros((4, lent))

    np.random.seed(seed)

    mu_true[:, 0] = mu0
    z_true[:, 0] = simulate_noisy_meas(mu0, R)

    for t in range(1, lent):
        mu_cur = mu_true[:, t - 1]
        # *****
        # State and Measurement Simulation *****
        # *****
        # TODO: implement the simulation of true state and measurements
        # Hint: use 'simulate_noisy_state_trans' and 'simulate_noisy_meas'

        mu_next = simulate_noisy_state_trans(mu_cur, u_cur, Q)
        z_next = simulate_noisy_meas(mu_next, R)

        mu_true[:, t] = mu_next
        z_true[:, t] = z_next

        # *****
        # *****

    return mu_true, z_true

# plot the estimated trajectory with truth
def plot_ekf_traj(mu_true, mu_hat, z_true, ax):

```

```

"""
Plot the EKF estimated trajectory with true trajectory
Inputs:
    mu_true: true trajectory (3 x lent)
    mu_hat: estimated trajectory (3 x lent)
    z_true: measurements (4 x lent)
    ax: matplotlib axis
Outputs:
    None
"""
xy_beacon = np.array([[-120, 120], [120, 120], [120, -120], [-120, -120]])

ax.plot(mu_true[0, :], mu_true[1, :], 'b-', label='true trajectory')
ax.scatter(mu_hat[0, :], mu_hat[1, :], 10, color='red', label='estimated_
↳trajectory')
ax.scatter(mu_true[0, 0], mu_true[1, 0], 50, marker='s', color='blue',
↳label='true initial position')
ax.scatter(mu_hat[0, 0], mu_hat[1, 0], 50, marker='s', color='red',
↳label='estimated initial position')
# plot beacons
for i in range(4):
    if i == 0:
        ax.plot(xy_beacon[i, 0], xy_beacon[i, 1], 'ko', label='beacons')
    else:
        ax.plot(xy_beacon[i, 0], xy_beacon[i, 1], 'ko')

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.axis('equal')
ax.grid(True)
ax.legend(loc='lower center')

def run_ekf_nominal_sim(env, plot=True):
    """
    Run the EKF simulation and plot results
    Inputs:
        plot: whether to plot the results
    Outputs:
        mu_hat: estimated trajectory (3 x lent)
        P_hat: estimated covariance (3 x 3 x lent)
    """
    mu0 = env['mu0']
    u_cur = env['u_cur']
    Q = env['Q']
    R = env['R']
    lent = env['lent']

```

```

# simulate true trajectory and measurements
n_mc = 4 # number of Monte Carlo simulations
P0 = np.diag(np.array([25, 25, 1]))
Q_ekf = np.diag(np.array([0.1, 0.1, 0.01]))
R_ekf = np.diag(np.array([10, 10, 10, 10]))

if plot:
    fig, axs = plt.subplots(2, 2, figsize=(10, 10))

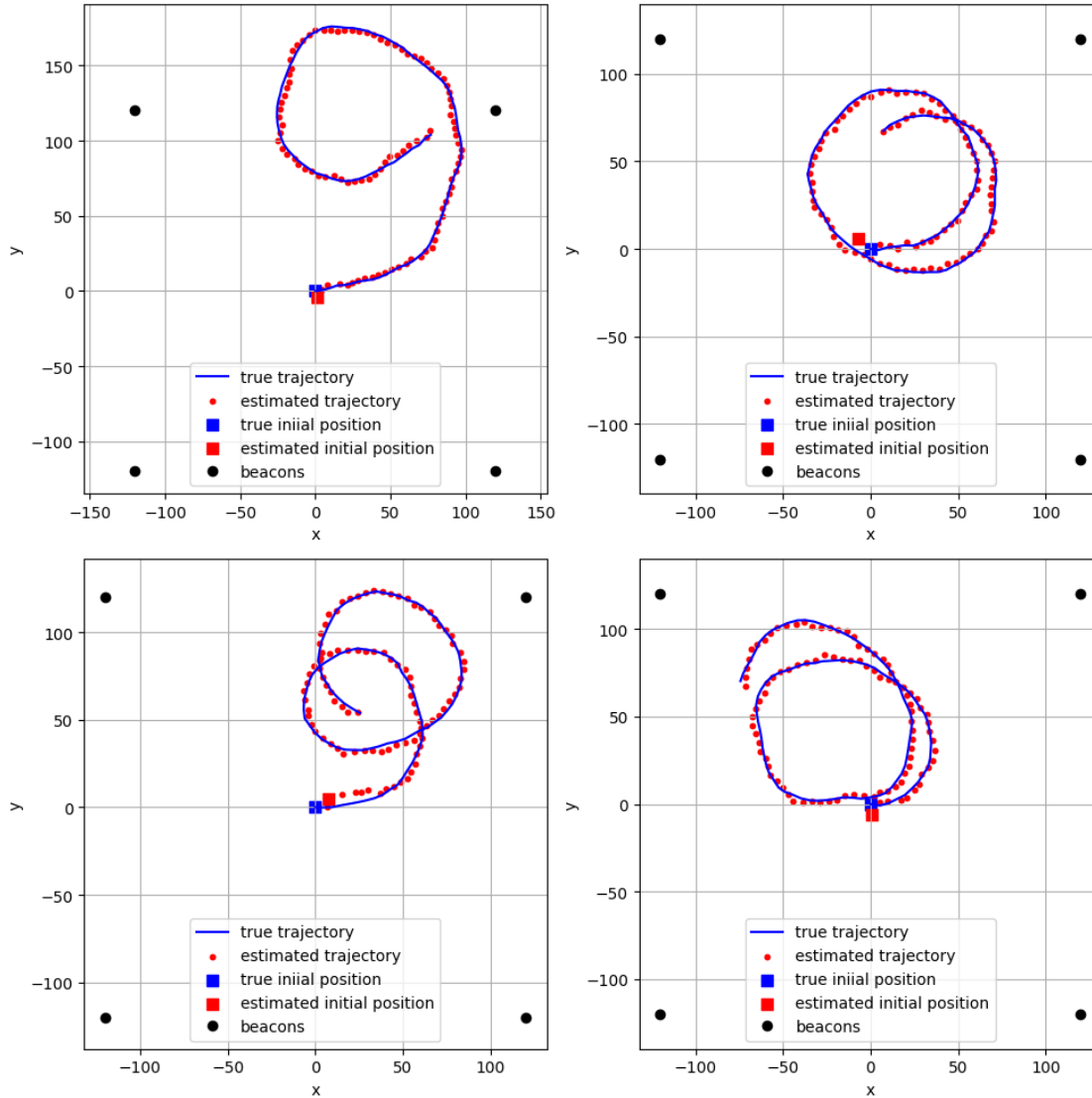
    for mci in range(n_mc):
        mu_true, z_true = sim_true_traj_meas(mu0, u_cur, Q, R, lent,
seed=env['seed'] + 2*mci)
        mu_hat, P_hat = run_ekf(Q_ekf, R_ekf, u_cur, mu_true, z_true, P0)
        if plot:
            plot_ekf_traj(mu_true, mu_hat, z_true, axs[mci // 2, mci % 2])

    if plot:
        plt.tight_layout()
        plt.show()

    return mu_hat, P_hat

mu_hat, P_hat = run_ekf_nominal_sim(env, plot=True)

```



```
[20]: # *** DO NOT MODIFY THE CODE BELOW ***
# For test (do not modify)
sol_37 = np.hstack([mu_hat.flatten(), P_hat.flatten()])
print("@P3.7\n", *np.round(sol_37, 2))
```

@P3.7

```
0.59 5.06 9.99 17.52 20.91 24.98 27.46 29.02 32.91 35.66 36.5 35.29 33.23 33.14
32.04 28.58 26.44 22.78 18.43 14.06 10.48 5.41 1.91 -2.44 -6.66 -11.66 -16.22
-21.29 -26.14 -31.47 -36.63 -41.71 -46.62 -51.16 -55.68 -59.05 -60.51 -63.59
-65.81 -67.42 -67.25 -65.72 -63.37 -62.83 -59.77 -58.31 -57.53 -53.3 -50.01
-47.02 -44.2 -38.71 -33.48 -28.67 -23.64 -18.91 -14.81 -9.7 -3.88 1.94 5.39 9.91
14.02 17.32 19.37 21.87 22.88 23.52 23.66 24.1 23.03 23.21 22.55 20.67 18.23
14.01 11.79 9.72 6.83 2.29 -2.06 -6.81 -9.69 -13.65 -18.6 -23.21 -28.31 -33.66
```

-37.74 -42.6 -48.49 -53.18 -57.19 -60.42 -65.04 -68.13 -69.53 -70.5 -71.2 -71.22
 -6.24 0.8 1.88 2.45 3.83 7.88 11.64 17.23 21.08 25.05 30.7 36.41 41.41 46.76
 51.37 55.37 59.96 63.66 66.73 69.59 71.33 72.51 76.37 79.18 81.89 82.29 82.77
 83.9 85.35 82.15 80.52 79.54 76.82 75.46 72.78 69.49 63.36 59.08 54.68 49.84
 44.99 40.17 35.28 30.16 25.98 21.61 16.31 13.16 9.61 5.69 1.97 1.29 1.81 0.79
 1.88 2.44 4.76 5.69 4.62 4.49 7.09 7.47 10.15 12.94 17.68 21.65 26.88 32.08
 37.18 42.16 47.64 52.91 57.05 61.9 66.72 69.96 74.94 79.61 82.91 85.79 88.66
 90.46 95.57 99.1 99.69 100.92 100.68 101.61 103.88 102.56 102.33 101.06 97.43
 93.63 91.42 88.56 82.93 77.45 72.57 67.28 0.64 1.04 0.61 0.28 0.44 0.68 0.86
 1.12 1.1 1.16 1.35 1.61 1.83 1.84 1.93 2.16 2.2 2.35 2.51 2.63 2.75 2.91 2.82
 2.84 2.86 3.02 3.13 3.16 3.17 3.45 3.54 3.57 3.68 3.71 3.8 3.94 4.26 4.29 4.38
 4.48 4.67 4.88 5.08 5.09 5.27 5.31 5.26 5.49 5.58 5.61 5.64 5.92 6.18 6.25 6.43
 6.53 6.72 6.73 6.6 6.57 6.77 6.76 6.89 7.02 7.26 7.38 7.58 7.73 7.86 7.93 8.07
 8.08 8.17 8.29 8.4 8.63 8.6 8.59 8.69 8.85 8.95 9.08 8.94 8.96 9.16 9.26 9.43
 9.46 9.43 9.61 9.64 9.76 10.0 10.2 10.18 10.3 10.57 10.77 10.9 11.03 25.0 4.3
 3.55 2.6 1.78 1.5 1.48 1.6 1.86 2.04 2.18 2.33 2.43 2.42 2.35 2.24 2.05 1.89 1.7
 1.5 1.31 1.13 0.97 0.9 0.85 0.82 0.75 0.69 0.67 0.65 0.68 0.77 0.85 0.95 1.04
 1.13 1.29 1.6 1.86 2.08 2.28 2.46 2.6 2.61 2.59 2.44 2.29 2.2 1.99 1.78 1.61
 1.48 1.24 1.0 0.87 0.81 0.82 0.9 0.98 0.96 0.92 0.98 1.02 1.13 1.29 1.58 1.86
 2.12 2.31 2.39 2.4 2.35 2.28 2.2 2.09 1.97 1.77 1.66 1.59 1.5 1.36 1.21 1.06
 1.03 1.01 0.91 0.81 0.72 0.68 0.66 0.66 0.68 0.73 0.87 1.09 1.24 1.39 1.65 1.95
 2.21 0.0 -0.18 -0.83 -1.05 -0.85 -0.78 -0.82 -0.86 -0.85 -0.83 -0.79 -0.66 -0.4
 -0.07 0.14 0.31 0.5 0.62 0.71 0.77 0.78 0.73 0.63 0.6 0.59 0.58 0.47 0.32 0.2
 0.12 -0.12 -0.31 -0.42 -0.52 -0.56 -0.6 -0.63 -0.61 -0.56 -0.49 -0.4 -0.23 0.03
 0.33 0.53 0.71 0.81 0.84 0.89 0.91 0.9 0.87 0.77 0.55 0.33 0.07 -0.16 -0.39
 -0.54 -0.57 -0.55 -0.64 -0.69 -0.77 -0.85 -0.89 -0.87 -0.74 -0.55 -0.32 -0.13
 0.07 0.19 0.31 0.44 0.57 0.7 0.78 0.83 0.86 0.88 0.87 0.8 0.82 0.83 0.75 0.62
 0.4 0.23 0.16 -0.01 -0.13 -0.27 -0.43 -0.54 -0.57 -0.57 -0.51 -0.38 -0.21 0.0
 -0.27 -0.48 -0.27 -0.13 -0.1 -0.1 -0.11 -0.14 -0.15 -0.15 -0.16 -0.16 -0.16
 -0.15 -0.14 -0.13 -0.12 -0.11 -0.1 -0.08 -0.07 -0.05 -0.05 -0.05 -0.05 -0.03
 -0.01 -0.0 0.0 0.03 0.05 0.06 0.07 0.08 0.09 0.1 0.12 0.14 0.15 0.15 0.16 0.17
 0.17 0.16 0.15 0.14 0.14 0.12 0.11 0.1 0.09 0.07 0.03 0.01 -0.01 -0.03 -0.06
 -0.07 -0.06 -0.05 -0.07 -0.07 -0.09 -0.1 -0.12 -0.14 -0.15 -0.16 -0.16 -0.15
 -0.15 -0.15 -0.14 -0.14 -0.13 -0.12 -0.11 -0.11 -0.1 -0.09 -0.08 -0.07 -0.07
 -0.07 -0.06 -0.04 -0.02 -0.0 0.0 0.02 0.03 0.04 0.07 0.09 0.1 0.11 0.12 0.14
 0.15 0.0 -0.18 -0.83 -1.05 -0.85 -0.78 -0.82 -0.86 -0.85 -0.83 -0.79 -0.66 -0.4
 -0.07 0.14 0.31 0.5 0.62 0.71 0.77 0.78 0.73 0.63 0.6 0.59 0.58 0.47 0.32 0.2
 0.12 -0.12 -0.31 -0.42 -0.52 -0.56 -0.6 -0.63 -0.61 -0.56 -0.49 -0.4 -0.23 0.03
 0.33 0.53 0.71 0.81 0.84 0.89 0.91 0.9 0.87 0.77 0.55 0.33 0.07 -0.16 -0.39
 -0.54 -0.57 -0.55 -0.64 -0.69 -0.77 -0.85 -0.89 -0.87 -0.74 -0.55 -0.32 -0.13
 0.07 0.19 0.31 0.44 0.57 0.7 0.78 0.83 0.86 0.88 0.87 0.8 0.82 0.83 0.75 0.62
 0.4 0.23 0.16 -0.01 -0.13 -0.27 -0.43 -0.54 -0.57 -0.57 -0.51 -0.38 -0.21 25.0
 4.41 2.86 2.75 2.78 2.57 2.25 1.93 1.57 1.37 1.22 1.05 0.88 0.84 0.85 0.91 1.05
 1.2 1.4 1.65 1.92 2.17 2.43 2.56 2.64 2.7 2.78 2.85 2.89 2.9 2.78 2.6 2.43 2.22
 2.04 1.87 1.67 1.37 1.17 1.02 0.9 0.79 0.74 0.8 0.87 1.02 1.14 1.17 1.33 1.5
 1.62 1.72 1.94 2.19 2.35 2.43 2.44 2.35 2.27 2.27 2.29 2.22 2.17 2.07 1.91 1.65
 1.42 1.17 0.98 0.87 0.81 0.81 0.82 0.86 0.94 1.06 1.3 1.46 1.56 1.68 1.89 2.13
 2.38 2.47 2.52 2.67 2.82 2.94 3.0 3.01 2.96 2.87 2.73 2.43 2.06 1.81 1.58 1.3

```

1.07 0.91 0.0 0.36 0.32 0.29 0.24 0.19 0.15 0.13 0.09 0.08 0.07 0.05 0.02 -0.02
-0.04 -0.05 -0.07 -0.09 -0.1 -0.12 -0.14 -0.15 -0.16 -0.17 -0.17 -0.17 -0.17 -0.17
-0.18 -0.18 -0.18 -0.17 -0.15 -0.14 -0.13 -0.12 -0.11 -0.1 -0.08 -0.06 -0.05
-0.04 -0.02 0.01 0.04 0.06 0.08 0.09 0.09 0.11 0.12 0.12 0.13 0.14 0.15 0.16
0.16 0.16 0.15 0.14 0.15 0.15 0.14 0.14 0.13 0.12 0.1 0.09 0.06 0.03 0.01 -0.01
-0.03 -0.03 -0.04 -0.06 -0.07 -0.1 -0.11 -0.11 -0.12 -0.13 -0.15 -0.16 -0.16
-0.16 -0.17 -0.18 -0.18 -0.18 -0.18 -0.18 -0.17 -0.16 -0.14 -0.11 -0.1 -0.09
-0.06 -0.04 -0.02 0.0 -0.27 -0.48 -0.27 -0.13 -0.1 -0.1 -0.11 -0.14 -0.15 -0.15
-0.16 -0.16 -0.16 -0.15 -0.14 -0.13 -0.12 -0.11 -0.1 -0.08 -0.07 -0.05 -0.05
-0.05 -0.05 -0.03 -0.01 -0.0 0.0 0.03 0.05 0.06 0.07 0.08 0.09 0.1 0.12 0.14
0.15 0.15 0.16 0.17 0.17 0.16 0.15 0.14 0.14 0.12 0.11 0.1 0.09 0.07 0.03 0.01
-0.01 -0.03 -0.06 -0.07 -0.06 -0.05 -0.07 -0.07 -0.09 -0.1 -0.12 -0.14 -0.15
-0.16 -0.16 -0.15 -0.15 -0.15 -0.14 -0.14 -0.13 -0.12 -0.11 -0.11 -0.1 -0.09
-0.08 -0.07 -0.07 -0.07 -0.06 -0.04 -0.02 -0.0 0.0 0.02 0.03 0.04 0.07 0.09 0.1
0.11 0.12 0.14 0.15 0.0 0.36 0.32 0.29 0.24 0.19 0.15 0.13 0.09 0.08 0.07 0.05
0.02 -0.02 -0.04 -0.05 -0.07 -0.09 -0.1 -0.12 -0.14 -0.15 -0.16 -0.17 -0.17
-0.17 -0.17 -0.18 -0.18 -0.18 -0.17 -0.15 -0.14 -0.13 -0.12 -0.11 -0.1 -0.08
-0.06 -0.05 -0.04 -0.02 0.01 0.04 0.06 0.08 0.09 0.09 0.11 0.12 0.12 0.13 0.14
0.15 0.16 0.16 0.16 0.15 0.14 0.15 0.15 0.14 0.14 0.13 0.12 0.1 0.09 0.06 0.03
0.01 -0.01 -0.03 -0.03 -0.04 -0.06 -0.07 -0.1 -0.11 -0.11 -0.12 -0.13 -0.15
-0.16 -0.16 -0.16 -0.17 -0.18 -0.18 -0.18 -0.18 -0.18 -0.17 -0.16 -0.14 -0.11
-0.1 -0.09 -0.06 -0.04 -0.02 1.0 0.56 0.2 0.09 0.05 0.04 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03
0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03 0.03

```

3.8 Changing Parameters for EKF [8 points]

In the previous part, we assumed we had access to the true Q and R covariance matrices. In other words, we gave our filter a perfect model of the process noise and measurement noise uncertainty. What if there is a mismatch between the true noise covariances and those used by our filter? Let's explore this further!

Let Q_{ekf} represent the state transition covariance used by the EKF, and let R_{ekf} represent the measurement covariance used by the EKF (which may be different from the *true* Q and R in your simulation). Repeat part (7) again with the following changes to the filter covariance matrices: 1. $Q_{\text{ekf}} = 0.01Q$ and $R_{\text{ekf}} = R$ 2. $Q_{\text{ekf}} = Q$ and $R_{\text{ekf}} = 0.01R$ 3. $Q_{\text{ekf}} = 0.01Q$ and $R_{\text{ekf}} = 0.01R$

Include your trajectory plot for each scaling scenario. Explain what differences, if any, you observe from your plot in part (7), and explain why these differences (or lack thereof) occur. **For fair comparison, make sure you use the same true trajectory and true, noisy measurements for all cases.**

Note: We are only changing the covariance matrices used by our EKF, denoted as Q_{ekf} and R_{ekf} .

ANSWER

[Provide three plots with different Q_{ekf} and R_{ekf} matrices (2 points)]

[Provide 2-3 sentence discussion on the differences you observe and an associated explanation for each of the three scenarios (6 points)]

```
[21]: def run_ekf_QR_sims(env):
    """
    Run EKF simulations with different Q and R settings
    Inputs:
        env: environment dictionary
    Outputs:
        None
    """
    mu0 = env['mu0']
    u_cur = env['u_cur']
    Q = env['Q']
    R = env['R']
    lent = env['lent']

    P0 = np.diag(np.array([25, 25, 1]))
    fig, axs = plt.subplots(2, 2, figsize=(10, 10))
    sim_num = 4

    Q_ekfs = [Q, 0.01 * Q, Q, 0.01 * Q]
    R_ekfs = [R, R, 0.01 * R, 0.01 * R]

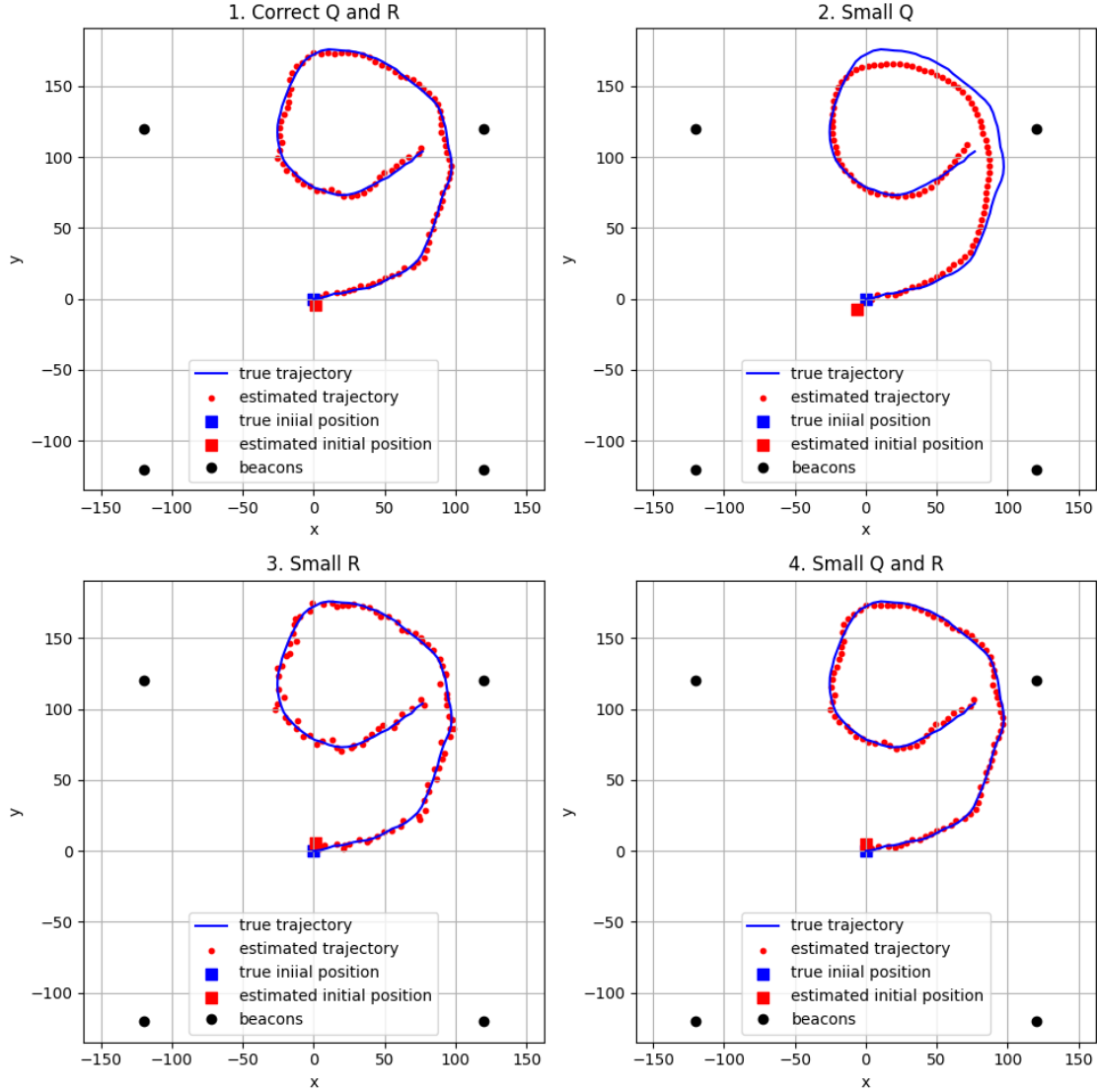
    condition = list(range(sim_num))
    condition[0] = '1. Correct Q and R'
    condition[1] = '2. Small Q'
    condition[2] = '3. Small R'
    condition[3] = '4. Small Q and R'

    # use the same true traj meas for all cases
    mu_true, z_true = sim_true_traj_meas(mu0, u_cur, Q, R, lent, seed=0)

    for si in range(sim_num):
        Q_ekf = Q_ekfs[si]
        R_ekf = R_ekfs[si]
        mu_hat, P_hat = run_ekf(Q_ekf, R_ekf, u_cur, mu_true, z_true, P0)
        plot_ekf_traj(mu_true, mu_hat, z_true, axs[si // 2, si % 2])
        axs[si // 2, si % 2].set_title(condition[si])

    plt.tight_layout()
    plt.show()

run_ekf_QR_sims(env)
```



For the case where $Q_{ekf} = 0.01Q$, the EKF assumes that the prediction is “perfect” because it propagates the model with no noise in the system dynamics. Therefore, it does not weight the measurements as high (sort of equivalent to dead reckoning). It is precise (due to propagating system dynamics), but not accurate.

For the case with only small R , we are assuming that the measurements are perfect with no noise, which weights them more in the final position estimate, which is why it aligns very closely with the true trajectory, but is very noisy. That is, the position estimate is accurate, but not precise.

Finally, with small Q and R , seems somewhat close to the final trajectory, just like with the full Q and R . This could be explained by the Q and R being scaled by the same amount, so they are “weighted” the same in the final estimate. This also means that the filter has high confidence in both its model and system dynamics.

Self-care

We are approaching the end of the autumn quarter (and this is your last homework for this class!) For many of us, end-of-the-quarter deadlines can be especially stressful. But, we also have the upcoming winter break to look forward to!

For this self-care practice, feel free to choose one of the following prompts to respond to:

- What is one thing, big or small, that you have recently done to take care of yourself during the end-of-the-quarter rush?
- After a particularly demanding period of time, when we ask a lot more of ourselves, it is vital to give ourselves extra time to rest and fully recover. In this respect, how do you plan to use the upcoming winter break to take extra good care of yourself? You all worked very hard this autumn quarter; you deserve it!

Describe your thoughts in a few sentences. Alternatively, feel free to be creative in your expression, including but not limited to a photo, video, or sketch.

ANSWER

I had 3 shows on Nov 7! First was at Hamm's Row Sig event with my more pop music band. Then immediately I went to Tressider to play a show with my pop punk / emo band. Finally, I went back to Hamm to play a complete second set with that first band. Although I was nervous for the first set, I thought the next 2 went so well, and I absolutely loved both of them (though prob the 2nd set because I absolutely love pop punk and emo music).